



دانشکده مهندسی کامپیوتر

طراحی و پیاده سازی سامانه هوشمند تشخیص دود و کنترل تجهیزات خانگی مبتنی بر ESP32

سامان صیاد فعال

۴۰۱۵۲۱۴۰۸

نام استاد:

دکتر امیر مهدی حسینی منزله

پروژه پایانی

شهریور ماه ۱۴۰۵

بِسْمِ اللَّهِ الرَّحْمَنِ الرَّحِيمِ

تشکر و قدردانی:

.....

چکیده

در این پروژه، یک سامانه هوشمند مبتنی بر ESP32 طراحی و پیاده سازی شده است که هدف اصلی آن تشخیص و اعلام شرایط خطر مانند دود و آتش و همچنین فراهم کردن قابلیت پایش محیط و کنترل تجهیزات الکترونیکی است.

دستگاه توسعه یافته علاوه بر قابلیت تشخیص دود و آتش، به حسگرهای دما و رطوبت و نور و تشخیص حرکت مجهز شده است تا بتواند اطلاعات مختلفی از وضعیت محیط را جمع آوری و پایش کند. همچنین با استفاده از گیرنده و فرستنده های مادون قرمز (IR) و رادیویی (RF)، امکان دریافت، ذخیره و ارسال فرمان های مربوط به تجهیزات الکتریکی فراهم شده است. کاربر می تواند از طریق سامانه نرم افزاری، وضعیت دستگاه و اطلاعات محیط را مشاهده کرده و تجهیزات را از راه دور روشن یا خاموش کرده و فرمان های مورد نظر را برای آن ها ارسال کند.

ارتباط دستگاه با سامانه نرم افزاری از طریق Wi-Fi برقرار شده و برای تبادل پیام و داده میان دستگاه و سامانه از پروتکل MQTT استفاده شده است.

واژه های کلیدی: خانه هوشمند، اتوماسیون خانگی، ESP32، اینترنت اشیا (IoT)

فهرست مطالب

1	فصل 1: کلیات پروژه
2	1-1- مقدمه
3	1-2- بیان مسئله
3	1-3- اهمیت و ضرورت پروژه
4	1-3-1- افزایش قابلیت پایش محیط
4	1-3-2- افزایش سرعت تشخیص رخدادها
4	1-3-3- امکان پایش از راه دور
4	1-3-4- یکپارچه سازی کنترل تجهیزات
5	1-3-5- کاهش نیاز به کنترل کننده های مجزا
5	1-3-6- قابلیت توسعه پذیری
5	1-4- اهداف پروژه
5	1-4-1- هدف اصلی
5	1-4-2- اهداف فرعی
6	1-5- محدوده پروژه
6	1-6- روش انجام پروژه
8	فصل 2: مبانی نظری
9	2-1- مقدمه
9	2-2- اینترنت اشیا
9	2-2-1- تعریف اینترنت اشیا
9	2-2-2- اجزای اصلی یک سامانه IoT
10	2-2-2-1- لایه حسگر
10	2-2-2-2- لایه پردازش
10	2-2-2-3- لایه ارتباطی
10	2-2-2-4- لایه کاربرد
10	2-2-3- کاربرد اینترنت اشیا در خانه هوشمند
11	2-3- سیستم های خانه هوشمند
11	2-3-1- مفهوم خانه هوشمند
11	2-3-2- پایش محیط
11	2-3-3- کنترل تجهیزات
12	2-4- میکروکنترلر ESP32
12	2-4-1- معرفی ESP32
12	2-4-2- ویژگی های ESP32
13	2-4-3- GPIO
13	2-4-4- مبدل آنالوگ به دیجیتال ADC
14	2-5- حسگر های محیطی
14	2-5-1- حسگر MQ-2
15	2-5-2- حسگر DHT11
15	2-5-2-2- نحوه ارتباط DHT11
15	2-5-3- حسگر شدت نور LDR
16	2-5-4- حسگر تشخیص حرکت PIR

فهرست مطالب

17	2-6- سیستم هشدار
17	2-6-1- بازر
17	2-7- ارتباط مادون قرمز
17	2-7-2- مفهوم IR
18	2-7-3- گیرنده IR
18	2-7-4- فرستنده IR
18	2-7-5- پروتکل های IR
18	2-7-6- کتابخانه IRremoteESP8266
19	2-8- ارتباط رادیویی RF
19	2-8-2- مفهوم RF
19	2-8-3- کتابخانه RCTrans
20	2-9- ارتباط Wi-Fi
20	2-9-1- معرفی Wi-Fi
20	2-9-2- حالت Station
20	2-9-3- حالت Access Point
20	2-9-4- حالت AP_STA
20	2-9-5- Captive Portal
21	2-10- پروتکل MQTT
21	2-10-1- معرفی MQTT
21	2-10-2- معماری Publish/Subscribe
21	2-10-3- MQTT Broker
21	2-10-4- Publisher
22	2-10-5- Subscriber
22	2-10-6- Topic/Payload
22	2-10-7- QoS
22	2-10-8- ارتباط MQTT از طریق WebSocket
23	2-11- ذخیره سازی اطلاعات در ESP32
23	2-11-1- حافظه Flash
23	2-11-2- LittleFS

فصل 3: تحلیل و طراحی سیستم

25	3-1- مقدمه
25	3-2- تحلیل نیازمندی های سیستم
25	3-2-1- نیازمندی های عملکردی
26	3-2-2- نیازمندی های غیر عملکردی
27	3-3- معماری کلی سیستم
27	3-3-2- لایه سخت افزار و جمع آوری داده
28	3-3-3- لایه پردازش و کنترل
28	3-3-4- لایه ارتباطی و رابط کاربر
28	3-4- طراحی سخت افزار
29	3-4-1- واحد پردازش مرکزی
29	3-4-2- سیستم اندازه گیری دما و رطوبت

فهرست مطالب

29	3-4-3 سیستم تشخیص دود و گاز.....
30	3-4-4 سیستم اندازه‌گیری شدت نور.....
30	3-4-5 سیستم تشخیص حرکت.....
30	3-4-6 سیستم ارتباط فرکانس رادیویی.....
31	3-4-7 سیستم ارتباط مادون قرمز.....
31	3-4-8 سیستم هشدار صوتی.....
31	3-4-9 LED وضعیت.....
31	3-5 پایه های ESP32.....
32	3-6 طراحی نرم افزار و ساختار Firmware.....
32	3-6-1 ساختار ماژولار Firmware.....
33	3-6-2 فرایند راه اندازی سیستم.....
33	3-6-3 حلقه اصلی اجرای برنامه.....
34	3-6-4 مدیریت حسگرها.....
34	3-6-5 مدیریت حسگر MQ-2.....
35	3-6-6 مدیریت حسگر DHT11.....
35	3-6-7 مدیریت حسگر PIR.....
35	3-6-8 مدیریت سیستم هشدار.....
36	3-7 طراحی ارتباط Wi-Fi و MQTT.....
36	3-7-1 ارتباط Wi-Fi.....
36	3-7-2 ارتباط MQTT.....
36	3-7-2-1 ساختار Topic های MQTT.....
36	3-7-2-2 Topic های مربوط به حسگرها.....
37	3-7-2-3 هشدار Topic.....
37	3-7-2-4 فرمان Topic.....
37	3-7-2-5 فهرست سیگنال های RF و IR.....
37	3-8 طراحی سیستم RF.....
37	3-8-1 دریافت سیگنال RF.....
38	3-8-2 ذخیره سیگنال RF.....
38	3-8-3 بازپخش سیگنال RF.....
38	3-8-4 مدیریت سیگنال های ذخیره شده RF.....
38	3-9 طراحی سیستم مادون قرمز.....
38	3-9-1 دریافت سیگنال IR.....
39	3-9-2 ذخیره سیگنال IR.....
39	3-9-3 بازپخش سیگنال IR.....
39	3-9-4 مدیریت سیگنال های ذخیره شده IR.....
39	3-10 طراحی سیستم ذخیره سازی.....
40	3-10-2 ذخیره تنظیمات Provisioning.....
40	3-10-3 ذخیره تنظیمات دستگاه.....
40	3-10-4 ساختار ذخیره سیگنال RF.....
41	3-10-5 ساختار ذخیره سیگنال IR.....
41	3-11 طراحی سیستم Provisioning.....
41	3-11-2 Captive Portal.....

فهرست مطالب

42	3-11-3 اتصال همزمان AP و Station
42	3-12 طراحی Web Dashboard
42	3-12-1 نمایش اطلاعات حسگرها
43	3-12-2 نمایش وضعیت هشدار
43	3-12-3 ارسال فرمان از طریق داشبورد
43	3-13 طراحی مدیریت خطا و ارتباطات
43	3-14 کتابخانه های نرم افزاری مورد استفاده

45 فصل 4: پیاده سازی سامانه

46	4-1 مقدمه
46	4-2 محیط و ابزار های توسعه
47	4-2-1 Framework تنظیمات برد و
47	4-2-2 تنظیمات ارتباط سریال
47	4-2-3 سیستم فایل LittleFS
47	4-2-4 فرایند ساخت و بارگذاری Firmware
48	4-3 پیاده سازی سخت افزار
48	4-3-1 معرفی ساختار سخت افزاری
48	4-3-2 پیاده سازی حسگر دما و رطوبت DHT11
49	4-3-3 پیاده سازی حسگر نور LDR
49	4-3-4 پیاده سازی حسگر MQ-2
49	4-3-5 پیاده سازی حسگر تشخیص حرکت PIR
50	4-3-6 پیاده سازی سیستم هشدار صوتی
51	4-4 پیاده سازی سیستم ارتباط رادیویی RF
51	4-4-1 معرفی سیستم RF
51	4-4-2 راه اندازی سیستم RF
51	4-4-3 ساختار داده سیگنال RF
52	4-4-4 دریافت سیگنال RF
52	4-4-5 مدیریت فرایند دریافت سیگنال
53	4-4-6 بازپخش سیگنال RF
53	4-4-7 مدیریت سیگنال های ذخیره شده
53	4-4-8 جست و جو سیگنال RF
54	4-4-9 حذف سیگنال ذخیره شده
54	4-4-10 منطق کلی دریافت و ارسال RF
55	4-5 پیاده سازی سیستم مادون قرمز IR
55	4-5-1 معرفی سیستم مادون قرمز
56	4-5-2 راه اندازی سیستم IR
56	4-5-3 ساختار داده سیگنال IR
57	4-5-4 دریافت و Decode سیگنال IR
57	4-5-5 مدیریت فرایند دریافت سیگنال
57	4-5-6 بازپخش سیگنال IR
57	4-5-7 مدیریت سیگنال های ذخیره شده
58	4-5-8 جست و جو سیگنال IR

فهرست مطالب

58	4-5-9 حذف سیگنال ذخیره شده
59	4-5-10 منطق کلی دریافت و ارسال IR
59	4-6 پیاده سازی ارتباط Wi-Fi و MQTT
59	4-6-1 مقدمه
60	4-6-2 ایجاد MQTT Client
60	4-6-3 برقراری ارتباط Wi-Fi
60	4-6-4 اتصال به MQTT Broker
61	4-6-5 راه اندازی اولیه MQTT
62	4-6-6 برقراری اتصال مجدد MQTT
63	4-6-7 دریافت فرمان های MQTT
63	4-6-8 انتشار اطلاعات در MQTT
64	4-7 پیاده سازی پیکربندی اولیه و مدیریت اتصال به شبکه
64	4-7-1 ساختار اطلاعات پیکربندی
65	4-7-2 AP اختصاصی برای پیکربندی
65	4-7-3 پورتال تحت وب پیکربندی
65	4-7-4 پیاده سازی Captive Portal
65	4-7-5 مدیریت درخواست های HTTP
66	4-7-6 دریافت و اعتبارسنجی تنظیمات
66	4-8 پیاده سازی مدیریت حافظه و ذخیره سازی اطلاعات
66	4-8-1 سیستم فایل LittleFS
66	4-8-2 قالب JSON
67	4-8-3 ذخیره و بازیابی وضعیت تشخیص حرکت
67	4-8-4 ذخیره و بازیابی سیگنال های RF
67	4-8-5 ذخیره و بازیابی سیگنال های IR
67	4-8-6 ذخیره و بازیابی تنظیمات Provisioning
68	4-9 راه اندازی و هماهنگی ماژول ها
68	4-9-1 تابع setup
68	4-9-2 تابع loop
69	4-9-3 راه اندازی ماژول های سخت افزاری
69	4-9-4 مدیریت فرمان های سیستم
69	4-9-4-1 پردازش فرمان های RF
70	4-9-4-2 پردازش فرمان های IR
70	4-9-4-3 مدیریت فرمان های سیستم هشدار
71	4-9-4-4 مدیریت فرمان های تشخیص حرکت
71	4-9-5 مدیریت خواندن حسگرها
72	4-9-5-1 زمان بندی خواندن حسگرها
72	4-10 پیاده سازی داشبورد وب و رابط کاربری
73	4-10-2 بخش تنظیمات ارتباط با MQTT
73	4-10-3 دریافت و پردازش داده های حسگرها
73	4-10-4 ارسال فرمان های کنترلی
73	4-10-5 سیستم هشدار داشبورد
73	4-10-6 ذخیره تنظیمات داشبورد

فهرست مطالب

74	فصل 5: آزمون و ارزیابی سیستم
75	5-1- مقدمه.....
75	5-2- روش آزمون و ارزیابی سامانه.....
75	5-2-1- محیط و تجهیزات مورد استفاده.....
75	5-2-2- معیار های ارزیابی.....
76	5-3- آزمون عملکرد حسگرها.....
76	5-3-1- آزمون حسگر دما و رطوبت.....
76	5-3-2- آزمون حسگر نور.....
77	5-3-3- آزمون حسگر دود و آتش.....
77	5-3-4- آزمون حسگر تشخیص حرکت.....
78	5-4- آزمون سامانه هشدار.....
78	5-5- آزمون اتصال به Wi-Fi و MQTT Broker.....
78	5-6- آزمون داشبورد سامانه.....
80	5-7- آزمون سیستم RF.....
80	5-7-1- آزمون دریافت سیگنال RF.....
80	5-7-2- آزمون ارسال سیگنال RF.....
80	5-7-3- آزمون مدیریت سیگنال RF.....
81	5-8- آزمون سیستم IR.....
81	5-8-1- آزمون دریافت سیگنال IR.....
81	5-8-2- آزمون ارسال سیگنال IR.....
82	5-8-3- آزمون مدیریت سیگنال IR.....
83	5-9- آزمون زیرسامانه Provisioning.....
83	5-10- جمع بندی فصل.....
84	فصل 6: نتیجه گیری
85	6-1- مقدمه.....
85	6-2- جمع بندی مراحل انجام پروژه.....
85	6-3- نتایج حاصل از پیاده سازی.....
86	6-4- محدودیت های پروژه.....
86	6-5- پیشنهاد های توسعه در آینده.....
86	6-5-1- افزایش امنیت ارتباطات.....
86	6-5-2- استفاده از Broker اختصاصی.....
86	6-5-3- توسعه قابلیت OTA.....
87	6-5-4- اضافه کردن ارتباطات جایگزین.....
87	6-6- جمع بندی نهایی.....
88	فصل 7: مراجع
90	فصل 8: پیوست ها
91	8-1- نقشه شماتیک مدار.....

فهرست اشکال

12	شکل (1-2) میکروکنترلر ESP32
14	شکل (2-2) حسگر MQ-2
15	شکل (3-2) حسگر DHT11
15	شکل (4-2) حسگر شدت نور
16	شکل (5-2) حسگر تشخیص حرکت PIR
17	شکل (6-2) بازر
17	شکل (7-2) گیرنده و فرستنده مادون قرمز
19	شکل (8-2) گیرنده رادیویی
19	شکل (9-2) فرستنده رادیویی
27	شکل (1-3) معماری کلی سیستم
33	شکل (2-3) روند راه اندازی سیستم
33	شکل (3-3) روند حلقه اصلی سیستم
34	شکل (4-3) روند مدیریت حسگر ها
34	شکل (5-3) روند مدیریت حسگر MQ-2
42	شکل (6-3) نمودار اتصال به شبکه
47	شکل (1-4) روند ساخت و بارگذاری Firmware
55	شکل (2-4) فرایند دریافت سیگنال RF
55	شکل (3-4) فرایند ارسال سیگنال RF
59	شکل (4-4) فرایند دریافت سیگنال IR
59	شکل (5-4) فرایند ارسال سیگنال IR
72	شکل (6-4) داشبورد وب
76	شکل (1-5) نتایج حسگر دما و رطوبت
76	شکل (2-5) نتیجه حسگر نور
77	شکل (3-5) نتایج حسگر دود و آتش
77	شکل (4-5) نتایج حسگر تشخیص حرکت
78	شکل (5-5) نتیجه آزمون اتصال به شبکه و MQTT Broker
79	شکل (6-5) نمایش اطلاعات حسگرها در داشبورد
79	شکل (7-5) نمایش هشدار در داشبورد
79	شکل (8-5) ارسال فرمان از داشبورد
80	شکل (9-5) دریافت سیگنال RF
80	شکل (10-5) ارسال سیگنال RF
81	شکل (11-5) مدیریت سیگنال RF
81	شکل (12-5) دریافت سیگنال IR
81	شکل (13-5) ارسال سیگنال IR
82	شکل (14-5) مدیریت سیگنال IR
83	شکل (15-5) صفحه تنظیمات Provisioning

فهرست جداول

31	جدول (1-3) پایه های ESP32
32	جدول (2-3) وظیفه مازول های Firmware
36	جدول (3-3) Topic های مربوط به حسگرها
39	جدول (4-3) فایل های اطلاعات دائمی سیستم
41	جدول (5-3) مشخصات AP
44	جدول (6-3) کتابخانه های مورد استفاده
52	جدول (1-4) ساختار داده سیگنال RF
56	جدول (2-4) ساختار داده سیگنال IR
64	جدول (3-4) ساختار داده تنظیمات پیکربندی

فصل 1:

کلیات پروژه

1-1- مقدمه

در سال‌های اخیر، توسعه فناوری‌های ارتباطی و سامانه‌های نهفته موجب گسترش استفاده از سیستم‌های هوشمند در محیط‌های مسکونی، اداری و صنعتی شده است. یکی از مهم‌ترین حوزه‌های حاصل از این توسعه، اینترنت اشیا (Internet of Things یا IoT) است. اینترنت اشیا به شبکه‌ای از تجهیزات فیزیکی اطلاق می‌شود که با بهره‌گیری از حسگرها، پردازنده‌ها، نرم‌افزارها و واسط‌های ارتباطی، قابلیت جمع‌آوری، پردازش، تبادل اطلاعات و در برخی موارد اعمال کنترل بر محیط پیرامون خود را دارند. [1] در چنین سامانه‌هایی، حسگرها وظیفه دریافت اطلاعات از محیط، واحد پردازشی وظیفه تحلیل و مدیریت داده‌ها و بخش ارتباطی وظیفه انتقال اطلاعات و دریافت فرمان‌های کنترلی را بر عهده دارد. [1] [2]

یکی از کاربردهای مهم اینترنت اشیا، توسعه سامانه‌های هوشمند خانگی است. در یک محیط هوشمند، تجهیزات مختلف می‌توانند از طریق یک شبکه ارتباطی با یکدیگر و با کاربر تعامل داشته باشند. این تعامل می‌تواند شامل پایش شرایط محیطی، تشخیص رخداد‌های غیرعادی، ارسال هشدار، کنترل تجهیزات الکتریکی و ثبت اطلاعات باشد. استفاده از چنین سامانه‌هایی می‌تواند امکان نظارت از راه دور و کاهش وابستگی به کنترل دستی تجهیزات را فراهم کند.

در این پروژه، یک هاب هوشمند مبتنی بر برد ESP32 طراحی و پیاده‌سازی شده است. ESP32 یک سامانه روی تراشه با قابلیت Wi-Fi در باند ۲.۴ گیگاهرتز، پردازنده ۳۲ بیتی و مجموعه‌ای از واسط‌های ورودی/خروجی دیجیتال و آنالوگ است که توسط شرکت Espressif ارائه شده و برای کاربردهایی از جمله تجهیزات هوشمند خانگی و اینترنت اشیا مناسب است. [3] این ویژگی‌ها باعث شده است ESP32 بتواند علاوه بر پردازش داده‌های حسگرها، ارتباط شبکه‌ای و کنترل تجهیزات جانبی را نیز در یک واحد سخت‌افزاری انجام دهد.

سیستم توسعه یافته در این پروژه علاوه بر تشخیص دود و گاز، امکان اندازه‌گیری دما، رطوبت و شدت نور و همچنین تشخیص حرکت را فراهم می‌کند. همچنین برای افزایش قابلیت کنترل تجهیزات مختلف، دو مجموعه ارتباط رادیویی در فرکانس‌های ۳۱۵ و ۴۳۳ مگاهرتز و یک مجموعه ارتباط مادون قرمز در نظر گرفته شده است. به این ترتیب، سیستم قادر است برخی سیگنال‌های رادیویی و مادون قرمز را دریافت، ذخیره و در صورت نیاز بازپخش کند.

ارتباط میان هاب هوشمند و زیرساخت نرم‌افزاری سیستم از طریق Wi-Fi و پروتکل MQTT انجام می‌شود. MQTT یک پروتکل ارتباطی سبک مبتنی بر الگوی انتشار/اشتراک (Publish/Subscribe) است که به طور ویژه برای محیط‌های دارای محدودیت منابع و کاربردهای ماشین به ماشین و اینترنت اشیا مناسب طراحی شده است. [4] در این پروژه، داده‌های حسگرها از طریق MQTT منتشر شده و فرمان‌های کنترلی نیز از طریق همین بستر به ESP32 ارسال می‌شوند.

از سوی دیگر، برای تعامل کاربر با سیستم، یک داشبورد تحت وب طراحی شده است که از طریق WebSocket به MQTT Broker متصل می‌شود. این داشبورد امکان مشاهده اطلاعات حسگرها، مشاهده هشدارها و ارسال فرمان‌های کنترلی را فراهم می‌کند. در نتیجه، پروژه حاضر را می‌توان یک هاب هوشمند چند منظوره دانست که وظایف پایش محیط، تشخیص رخداد، ایجاد هشدار، ارتباط شبکه‌ای و کنترل تجهیزات را در یک سامانه یکپارچه انجام می‌دهد.

1-2- بیان مسئله

در بسیاری از محیط‌های مسکونی و اداری، تجهیزات الکتریکی و سامانه های مختلف به صورت مستقل مورد استفاده قرار می‌گیرند و اطلاعات مربوط به شرایط محیطی نیز معمولاً به صورت متمرکز و مستمر جمع آوری نمی‌شود. در روش های سنتی، تشخیص رخدادهایی مانند افزایش میزان دود یا گاز، تغییرات دما، حضور افراد یا تغییر شدید شرایط نوری ممکن است به صورت دستی یا توسط تجهیزات مستقل انجام شود. چنین رویکردی علاوه بر افزایش پیچیدگی سیستم، امکان پایش یکپارچه و کنترل از راه دور را محدود می‌کند.

از طرف دیگر، بسیاری از تجهیزات موجود در محیط‌های خانگی از روش های ارتباطی متفاوتی برای کنترل استفاده می‌کنند. برخی تجهیزات از ریموت های مادون قرمز و برخی دیگر از ریموت های رادیویی در فرکانس هایی مانند ۳۱۵ یا ۴۳۳ مگاهرتز استفاده می‌کنند. در نتیجه، برای کنترل یکپارچه این تجهیزات معمولاً نیاز به چندین کنترل کننده و واسط مجزا وجود دارد.

مسئله اصلی این پروژه، طراحی و پیاده‌سازی یک سامانه یکپارچه است که بتواند اطلاعات مربوط به شرایط محیط را از طریق مجموعه‌ای از حسگرها دریافت کرده، آن‌ها را در یک واحد پردازشی مدیریت کند، اطلاعات را از طریق شبکه Wi-Fi و MQTT در اختیار نرم‌افزار قرار دهد و امکان کنترل تجهیزات الکتریکی را نیز فراهم کند.

برای دستیابی به این هدف، از ESP32 به عنوان هسته پردازشی و ارتباطی سیستم استفاده شده است. در بخش پایش محیط، سنسور MQ-2 برای تشخیص دود و گازهای قابل اشتعال، سنسور DHT11 برای اندازه گیری دما و رطوبت، حسگر نوری LDR برای اندازه گیری نسبی شدت نور و حسگر حرکتی PIR برای تشخیص حرکت مورد استفاده قرار گرفته‌اند. اطلاعات به دست آمده از این حسگرها به صورت دوره‌ای پردازش و از طریق MQTT منتشر می‌شوند.

در بخش کنترل نیز سیستم از گیرنده و فرستنده های RF در دو باند ۳۱۵ و ۴۳۳ مگاهرتز و همچنین گیرنده و فرستنده مادون قرمز استفاده می‌کند. کتابخانه RCSSwitch امکان دریافت و ارسال کدهای برخی تجهیزات رادیویی رایج در فرکانس های ۳۱۵ و ۴۳۳ مگاهرتز را فراهم می‌کند. [5] همچنین کتابخانه IRremoteESP8266 برای دریافت و ارسال سیگنال های مادون قرمز مورد استفاده قرار گرفته است. [6] بنابراین، مسئله این پروژه را می‌توان به صورت طراحی یک هاب هوشمند تعریف کرد که سه وظیفه اصلی را در یک بستر واحد ترکیب می‌کند:

- پایش شرایط محیطی
- تشخیص رخداد و ایجاد هشدار
- کنترل تجهیزات الکتریکی و دریافت/بازپخش فرمان های بی‌سیم

1-3- اهمیت و ضرورت پروژه

افزایش استفاده از تجهیزات الکتریکی و گسترش سیستم های هوشمند، نیاز به سامانه هایی را افزایش داده است که بتواند علاوه بر کنترل تجهیزات، وضعیت محیط را نیز پایش کنند. یکی از مزیت های اصلی استفاده از اینترنت اشیا در چنین سامانه هایی، ایجاد ارتباط میان دنیای فیزیکی و سامانه های نرم افزاری است؛ به گونه ای که اطلاعات حسگر ها می‌توانند دریافت، منتقل و در بخش نرم افزاری پردازش شوند.

اهمیت این پروژه را می توان از جنبه های مختلف بررسی کرد.

1-3-1- افزایش قابلیت پایش محیط

وجود چندین حسگر در یک سامانه امکان جمع آوری همزمان اطلاعات مختلف محیطی را فراهم می کند. در پروژه حاضر، اطلاعات مربوط به دما، رطوبت، شدت نور، میزان دود و وضعیت حرکت در اختیار سیستم قرار می گیرد. این اطلاعات می توانند برای بررسی وضعیت لحظه ای محیط و تشخیص برخی شرایط غیرعادی مورد استفاده قرار گیرند.

1-3-2- افزایش سرعت تشخیص رخدادها

استفاده از سنسور های محیطی امکان واکنش سریع تر سیستم نسبت به رخدادهایی مانند افزایش مقدار گاز یا دود و تشخیص حرکت را فراهم می کند. در پروژه حاضر، در صورت عبور مقدار خوانده شده از سنسور MQ-2 از آستانه تعیین شده، سیستم وضعیت هشدار را فعال کرده و پیام مربوطه را از طریق MQTT منتشر می کند. همچنین در صورت تشخیص حرکت، امکان ایجاد هشدار و فعال سازی بازو وجود دارد.

1-3-3- امکان پایش از راه دور

ارتباط Wi-Fi و MQTT باعث می شود اطلاعات جمع آوری شده تنها در سطح خود دستگاه باقی نماند و بتوان آن ها را به یک Broker و نرم افزارهای دیگر منتقل کرد. MQTT به دلیل استفاده از مدل Publish/Subscribe، امکان جداسازی تولیدکننده و مصرف کننده داده را فراهم می کند و برای ارتباط میان تجهیزات IoT مناسب است. [4]

1-3-4- یکپارچه سازی کنترل تجهیزات

یکی از ویژگی های مهم پروژه، ترکیب پایش محیط با کنترل تجهیزات الکتریکی است. سیستم می تواند سیگنال های RF در دو باند ۳۱۵ و ۴۳۳ مگاهرتز را دریافت و ذخیره کرده و در زمان مناسب بازپخش کند. کتابخانه RCSSwitch نیز برای کنترل تجهیزات رادیویی رایج با فرستنده ها و گیرنده های ۳۱۵ و ۴۳۳ مگاهرتز طراحی شده است. [5]

همچنین استفاده از ارتباط مادون قرمز باعث می شود سیستم بتواند با برخی تجهیزات مجهز به ریموت IR تعامل داشته باشد. کتابخانه IRremoteESP8266 برای ESP32 امکان دریافت و ارسال سیگنال های مادون قرمز و پشتیبانی از پروتکل های متعدد را فراهم می کند. [6]

5-3-1- کاهش نیاز به کنترل کننده های مجزا

ترکیب چندین قابلیت در یک برد ESP32 باعث می شود به جای استفاده از چند سیستم مستقل برای پایش محیط، دریافت فرمان های بی سیم و کنترل تجهیزات، بتوان بخش قابل توجهی از این وظایف را در یک هاب واحد پیاده سازی کرد.

6-3-1- قابلیت توسعه پذیری

معماری ماژولار نرم افزار، استفاده از MQTT و قابلیت های ارتباطی ESP32 امکان توسعه سیستم در آینده را فراهم می کند. برای مثال، می توان حسگرهای جدید، تجهیزات کنترلی بیشتر یا سرویس های نرم افزاری دیگر را بدون تغییر اساسی در ساختار کلی سیستم به آن اضافه کرد.

4-1- اهداف پروژه

1-4-1- هدف اصلی

هدف اصلی این پروژه، طراحی و پیاده سازی یک سامانه هوشمند مبتنی بر ESP32 برای تشخیص دود و آتش و پایش محیط و کنترل تجهیزات الکتریکی از طریق شبکه Wi-Fi است.

2-4-1- اهداف فرعی

اهداف فرعی پروژه عبارتند از:

1. طراحی و پیاده سازی بخش سخت افزاری مبتنی بر ESP32
2. تشخیص وجود دود و شرایط مرتبط با آتش با استفاده از MQ-2
3. اندازه گیری دما و رطوبت محیط با استفاده از DHT11
4. اندازه گیری شدت نور محیط با استفاده از LDR
5. تشخیص حضور یا حرکت در محیط با استفاده از حسگر PIR
6. ایجاد سیستم هشدار صوتی با استفاده از بازر
7. دریافت سیگنال های مادون قرمز از کنترلر تجهیزات مختلف
8. ذخیره فرمان های مادون قرمز دریافت شده

9. ارسال سیگنال های مادون قرمز برای کنترل تجهیزات مختلف
10. دریافت و ارسال سیگنال های رادیویی برای کنترل تجهیزات رادیویی
11. برقراری ارتباط دستگاه با شبکه Wi-Fi
12. استفاده از MQTT برای تبادل اطلاعات میان دستگاه و سامانه نرم افزاری
13. ایجاد قابلیت پایش سیستم از راه دور
14. فراهم کردن امکان کنترل تجهیزات الکتریکی از طریق سامانه نرم افزاری
15. ایجاد ساختاری قابل توسعه برای اضافه کردن حسگرها و قابلیت های جدید در آینده

5-1- محدوده پروژه

محدوده این پروژه شامل طراحی و پیاده سازی یک نمونه عملیاتی از سامانه هوشمند پایش و کنترل محیط مبتنی بر ESP32 است. در این پروژه، ESP32 به عنوان واحد پردازش مرکزی وظیفه دریافت اطلاعات از حسگرها، پردازش داده ها، مدیریت ارتباطات و ارسال یا دریافت سیگنال های مادون قرمز و رادیویی را بر عهده دارد.

- تشخیص دود و شرایط مرتبط با آتش
 - اندازه گیری دما و رطوبت و شدت نور
 - تشخیص حرکت
 - ارسال و دریافت سیگنال های مادون قرمز
 - ارسال و دریافت سیگنال های رادیویی
 - اتصال دستگاه به Wi-Fi
 - ارتباط با سامانه نرم افزاری
 - ارسال اطلاعات حسگرها به سامانه نرم افزاری
 - دریافت فرمان از سامانه نرم افزاری برای کنترل تجهیزات الکتریکی
- با وجود قابلیت های فوق، پروژه دارای محدوده مشخصی است. برای مثال، عملکرد تشخیص آتش در این پروژه در سطح یک سامانه نمونه و آموزشی/مهندسی در نظر گرفته شده و نباید آن را به عنوان جایگزین تجهیزات ایمنی و اعلام حریق دارای تاییدیه استاندارد برای کاربرد های حیاتی در نظر گرفت. همچنین کنترل تجهیزات از طریق IR و RF به نوع پروتکل و سازگاری تجهیزات مقصد وابسته است.

6-1- روش انجام پروژه

فرایند انجام پروژه به صورت مرحله ای و بر اساس نیازمندی های سخت افزاری و نرم افزاری سیستم انجام شده است. در مرحله نخست، مسئله و نیازمندی های اصلی سیستم شامل تشخیص شرایط خطر، پایش محیط و کنترل تجهیزات الکتریکی بررسی شد. سپس معماری کلی سیستم و نحوه ارتباط میان حسگرها، ESP32 و سامانه نرم افزاری مشخص گردید.

در مرحله بعد، قطعات سخت افزاری مورد نیاز انتخاب و مدار اولیه طراحی شد. ESP32 به عنوان کنترل کننده مرکزی انتخاب شد؛ زیرا علاوه بر قابلیت پردازشی، امکانات ارتباطی مورد نیاز پروژه از جمله Wi-Fi را در اختیار قرار می دهد.

پس از طراحی سخت افزار، Firmware دستگاه توسعه داده شد. این بخش وظیفه خواندن اطلاعات حسگرها، تشخیص وضعیت های مختلف، مدیریت ورودی و خروجی ها، دریافت و ارسال سیگنال های IR و RF و همچنین مدیریت ارتباط شبکه را بر عهده دارد.

در بخش ارتباطی، دستگاه از طریق Wi-Fi به شبکه متصل شده و ارتباط میان ESP32 و سامانه نرم افزاری با استفاده از MQTT برقرار شده است. در این معماری، دستگاه می تواند اطلاعات مربوط به حسگرها و وضعیت خود را در Topic های مشخص منتشر کند و در مقابل، پیام های کنترلی را از Topic های مربوطه دریافت نماید.

در ادامه بخش نرم افزاری توسعه داده شده و قابلیت مشاهده اطلاعات و ارسال فرمان های کنترلی در آن فراهم شده است.

پس از اتصال اجزای مختلف، آزمایش های مربوط به حسگرها، ارتباط Wi-Fi، تبادل پیام MQTT و عملکرد فرستنده و گیرنده های IR و RF انجام شده است.

در نهایت، عملکرد کلی سامانه با بررسی سناریو های مختلف مورد ارزیابی قرار گرفته است.

فصل 2:

مبانی نظری

1-2- مقدمه

گسترش فناوری های ارتباطی، میکروکنترلر های کم مصرف و شبکه های بی سیم باعث شده است که تجهیزات الکترونیکی بتوانند علاوه بر انجام وظایف کنترلی، اطلاعات محیط پیرامون خود را نیز دریافت و از طریق شبکه منتقل کنند. این رویکرد یکی از پایه های اصلی شکل گیری اینترنت اشیا (Internet of Things یا IoT) است.

در سامانه های هوشمند، ترکیب حسگر ها، واحد پردازش، ارتباط شبکه و نرم افزار کاربردی امکان پایش شرایط محیطی و کنترل تجهیزات را فراهم می کند. در یک خانه هوشمند، چنین سامانه ای می تواند اطلاعاتی مانند دما، رطوبت، شدت نور، وجود دود یا آتش را دریافت کرده و در مقابل، تجهیزات مختلف را از طریق ارتباطات بی سیم کنترل کند.

پروژه حاضر یک سیستم هوشمند مبتنی بر ESP32 است که علاوه بر پایش شرایط محیطی، قابلیت تشخیص دود و آتش، تشخیص حرکت، ایجاد هشدار صوتی و دریافت و ارسال فرمان های مادون قرمز و رادیویی را فراهم می کند. ارتباط دستگاه با زیرساخت شبکه از طریق Wi-Fi و تبادل داده با استفاده از پروتکل MQTT انجام می شود.

2-2- اینترنت اشیا

1-2-2- تعریف اینترنت اشیا

اینترنت اشیا به مجموعه ای از تجهیزات فیزیکی گفته می شود که دارای حسگر، قابلیت پردازش و ارتباط شبکه بوده و می توانند اطلاعات مربوط به محیط یا وضعیت خود را جمع آوری و با سایر سامانه ها مبادله کنند. [1]

در یک سامانه IoT معمولاً سه وظیفه اصلی وجود دارد:

- دریافت اطلاعات از محیط
- پردازش و تفسیر اطلاعات
- ارسال اطلاعات یا اجرای فرمان

در پروژه حاضر، حسگرهای MQ-2، DHT11، LDR و PIR وظیفه دریافت اطلاعات محیطی را بر عهده دارند. ESP32 وظیفه پردازش اولیه را انجام داده و Wi-Fi و MQTT برای انتقال اطلاعات به کار می روند.

2-2-2- اجزای اصلی یک سامانه IoT

یک سامانه اینترنت اشیا را می توان به چند بخش تقسیم کرد:

1-2-2-2-2-1 لایه حسگر

در این لایه، اطلاعات از محیط دریافت می شود. حسگر می تواند کمیتی فیزیکی مانند دما، رطوبت، نور، حرکت یا غلظت گاز را به سیگنال الکتریکی تبدیل کند. در پروژه حاضر این لایه شامل موارد زیر است:

- DHT11
- LDR
- MQ-2
- PIR

2-2-2-2-2-2 لایه پردازش

در این لایه، داده های دریافت شده، پردازش شده و تصمیم های لازم گرفته می شود. ESP32 هسته اصلی پردازش پروژه است.

3-2-2-2-2-3 لایه ارتباطی

این لایه وظیفه انتقال داده بین دستگاه و سایر اجزای سامانه را بر عهده دارد. در این پروژه از Wi-Fi برای ارتباط با شبکه و از پروتکل MQTT برای انتقال داده استفاده شده است.

4-2-2-2-2-4 لایه کاربرد

در این لایه، اطلاعات به کاربر نمایش داده شده و فرمان های کنترلی دریافت می شوند. داشبورد وب پروژه از طریق MQTT over WebSocket با Broker ارتباط برقرار می کند.

3-2-2-2- کاربرد اینترنت اشیا در خانه هوشمند

خانه هوشمند یکی از کاربردهای مهم IoT است. در چنین سامانه ای، تجهیزات مختلف می توانند به شبکه متصل شده و از طریق یک سیستم مرکزی کنترل یا پایش شوند. برخی کاربردهای رایج عبارتند از:

- کنترل روشنایی
- کنترل لوازم الکتریکی
- پایش دما و رطوبت
- تشخیص حرکت
- تشخیص دود و گاز
- سیستم های هشدار
- کنترل تجهیزات صوتی و تصویری
- مدیریت مصرف انرژی

در پروژه حاضر، ترکیب پایش محیطی و کنترل تجهیزات باعث شده است که دستگاه علاوه بر نقش یک سنسور، به عنوان یک هاب کنترل کننده و اتوماسیون خانه نیز عمل کند.

2-3- سیستم های خانه هوشمند

2-3-1- مفهوم خانه هوشمند

خانه هوشمند محیطی است که در آن تجهیزات مختلف از طریق فناوری های ارتباطی به یکدیگر متصل شده و امکان پایش و کنترل آن ها به صورت خودکار از راه دور فراهم می شود. در یک سیستم خانه هوشمند، ارتباط میان تجهیزات می تواند با فناوری های مختلفی مانند Zigbee، Bluetooth، Wi-Fi، Z-Wave یا فناوری های مبتنی بر اینترنت اشیا انجام شود. در پروژه حاضر، برای ارتباط دستگاه با شبکه از Wi-Fi استفاده شده و برای کنترل تجهیزات از راه دور از IR و RF استفاده می شود.

2-3-2- پایش محیط

پایش محیط به فرایند اندازه گیری مداوم یا دوره ای پارامتر های محیطی گفته می شود. پارامتر های مورد استفاده در این پروژه عبارتند از:

- دما
- رطوبت
- شدت نور
- شدت گاز های موجود در محیط
- وضعیت حرکت و حضور

داده های حاصل از این حسگر ها در فواصل زمانی مشخص خوانده و از طریق MQTT منتشر می شوند.

2-3-3- کنترل تجهیزات

یکی از قابلیت های مهم خانه هوشمند، کنترل تجهیزات الکتریکی بدون نیاز به استفاده مستقیم از کنترل کننده اصلی آن است.

در پروژه حاضر، برای این منظور دو روش اصلی وجود دارد:

- مادون قرمز (IR):

فرستادن سیگنال: دستگاه $ESP32 \rightarrow IR LED$

گرفتن سیگنال: $ESP32 \rightarrow IR Receiver$ → کنترل دستگاه

- فرکانس رادیویی (RF):

فرستادن سیگنال: دستگاه $ESP32 \rightarrow RF Transmitter$

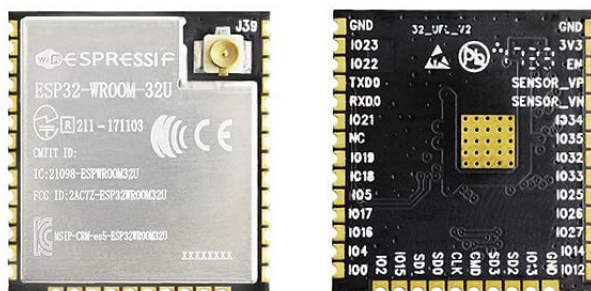
گرفتن سیگنال: $ESP32 \rightarrow RF Receiver$ → ریموت دستگاه

2-4- میکروکنترلر ESP32

2-4-1 معرفی ESP32

ESP32 یک خانواده از SoC های شرکت Espressif Systems است که برای کاربرد های ارتباطی و سیستم های نهفته طراحی شده است. این خانواده دارای پردازنده ۳۲ بیتی، حافظه، Wi-Fi، ADC، Bluetooth و مجموعه ای از رابط های جانبی است. طبق مستندات رسمی، ESP32 از Wi-Fi با استاندارد های 802.11b/g/n پشتیبانی کرده و دارای ADC دوازده بیتی، GPIO های متعدد، PWM، I2C، SPI و سایر واحد های جانبی است. [3]

وجود Wi-Fi داخلی یکی از دلایل اصلی انتخاب ESP32 برای این پروژه است؛ زیرا دستگاه می تواند بدون نیاز به ماژول ارتباطی جداگانه به شبکه متصل شود.



شکل (2-1) میکروکنترلر ESP32

2-4-2 ویژگی های ESP32

برخی ویژگی های مهم ESP32 عبارتند از: [3]

- پردازنده ۳۲ بیتی Xtensa LX6
- Wi-Fi 802.11 b/g/n
- Bluetooth
- GPIO قابل برنامه ریزی
- ADC
- DAC
- PWM
- UART
- SPI
- I2C

- RMT
- تایمر
- حافظه SRAM
- حافظه Flash

GPIO -2-4-3

GPIO یا General Purpose Input/Output پایه های قابل برنامه ریزی میکروکنترلر هستند که می توانند برای دریافت یا ارسال سیگنال دیجیتال استفاده شوند. در این پروژه از GPIO ها برای اتصال تجهیزاتی مانند:

- PIR
 - Buzzer
 - IR LED
 - IR Receiver
 - RF Transmitter
 - RF Receiver
- استفاده شده است.

ADC -2-4-4 مبدل آنالوگ به دیجیتال

ADC یا Analog-to-Digital Converter وظیفه تبدیل یک سیگنال ولتاژ آنالوگ به مقدار دیجیتال قبل پردازش توسط میکروکنترلر را بر عهده دارد. در پروژه حاضر خروجی آنالوگ دو حسگر اصلی به ADC متصل شده است:

- LDR
- MQ-2

در یک ADC دوازده بیتی مقدار دیجیتال تئوری در محدوده:

$$0 \leq ADC \leq 4095$$

قرار می گیرد.

2-5- حسگر های محیطی

2-5-1- حسگر MQ-2



شکل (2-2) حسگر MQ-2

MQ-2 یک حسگر گاز از خانواده حسگر های گازی مقاومتی است که برای تشخیص گاز های قابل اشتعال و دود استفاده می شود.

مطابق اطلاعات فنی منتشر شده برای MQ-2، این حسگر برای تشخیص موادی مانند:

- LPG
- متان
- پروپان
- بوتان
- الکل
- هیدروژن
- دود

کاربرد دارد.[7]

عملکرد حسگر بر اساس تغییر مقاومت ماده حساس به گاز در حضور گاز های موردنظر است. تغییر مقاومت باعث تغییر سیگنال خروجی می شود که می توان آن را توسط ADC میکرو کنترلر اندازه گیری کرد.

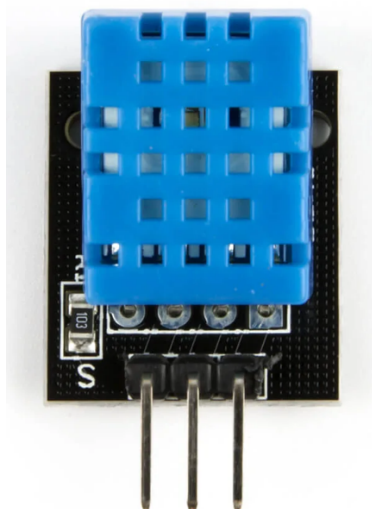
پیش گرمایش MQ-2

حسگر های خانواده MQ برای رسیدن به وضعیت کاری پایدار به زمان گرم شدن نیاز دارند و مقدار آن بلافاصله بعد از روشن شدن برای تصمیم گیری استفاده نمی شود و یک دوره پیش گرمایش در نظر گرفته می شود.[7]

محدودیت MQ-2

MQ-2 یک حسگر عمومی تشخیص گاز و دود است و نباید آن را به تنهایی معادل یک سیستم استاندارد تشخیص و اعلام حریق در نظر گرفت. مقدار خوانده شده از این حسگر به شرایط محیطی، گاز مورد نظر، دما، رطوبت، زمان گرم شدن و ساختار مدار وابسته است.

2-5-2- حسگر DHT11



شکل (2-3) حسگر DHT11

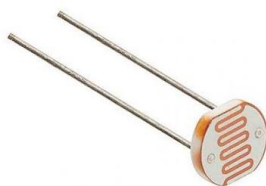
DHT11 یک حسگر دیجیتال برای اندازه‌گیری هم زمان دما و رطوبت نسبی است و داده‌های اندازه‌گیری شده DHT11 با یک پروتکل دیجیتال به میکروکنترلر منتقل می‌شود.

2-5-2-2 نحوه ارتباط DHT11

ارتباط DHT11 با میکروکنترلر بر پایه یک رابط تک سیمی و زمان بندی مشخص انجام می‌شود. میکروکنترلر درخواست اندازه‌گیری را ارسال کرده و سپس داده‌های مربوط به دما و رطوبت را دریافت می‌کند.

کتابخانه Adafruit DHT امکان ایجاد شی DHT و خواندن مستقیم دما و رطوبت را فراهم می‌کند. مستندات رسمی این کتابخانه توابعی مانند `readTemperature()` و `readHumidity()` را ارائه می‌دهد. [8]

2-5-3- حسگر شدت نور LDR



شکل (2-4) حسگر شدت نور

LDR یا Light Dependant Resistor یک مقاومت وابسته به نور است که مقدار مقاومت الکتریکی آن با شدت نور تغییر می کند. در حالت کلی با افزایش شدت نور، مقاومت LDR تغییر می کند و با قرار دادن آن در یک مدار تقسیم ولتاژ می توان تغییر مقاومت را به تغییر ولتاژ تبدیل کرد.

2-5-4- حسگر تشخیص حرکت PIR



شکل (2-5) حسگر تشخیص حرکت PIR

حسگر PIR یا Passive Infrared Sensor برای تشخیص تغییرات تابش مادون قرمز محیط استفاده می شود. بدن انسان و سایر اجسام گرم تابش مادون قرمز منتشر می کنند. هنگامی که یک جسم گرم در محدوده دید حسگر حرکت می کند، تغییر تابش دریافتی توسط حسگر تشخیص داده شده و ماژول خروجی دیجیتالی تولید می کند.

2-6- سیستم هشدار

2-6-1- بازر



شکل (2-6) بازر

بازر یک مبدل الکتریکی به صوتی است که با دریافت سیگنال مناسب، صدای هشدار ایجاد می کند.

2-7- ارتباط مادون قرمز



شکل (2-7) گیرنده و فرستنده مادون قرمز

2-7-2- مفهوم IR

مادون قرمز یا Infrared بخشی از طیف الکترومغناطیسی است که طول موج آن بیشتر از نور قرمز مرئی است.

در سیستم های کنترل از راه دور، اطلاعات دیجیتال با تغییرات زمانی تابش IR منتقل می شوند. کنترل تلویزیون، سیستم تهویه، پخش کننده ها و بسیاری از تجهیزات خانگی از این روش استفاده می کنند.

2-7-3- گیرنده IR

گیرنده IR وظیفه دریافت تابش مادون قرمز و تبدیل آن به سیگنال الکتریکی را بر عهده دارد.

2-7-4- فرستنده IR

فرستنده IR وظیفه تبدیل سیگنال الکتریکی به تابش مادون قرمز و ارسال آن را بر عهده دارد.

2-7-5- پروتکل های IR

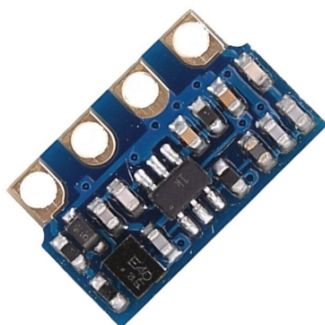
ریموت های مختلف از پروتکل های مختلفی برای انتقال فرمان استفاده می کنند. از پروتکل های رایج می توان به :

- NEC
 - Sony
 - Samsung
 - Panasonic
- اشاره کرد.

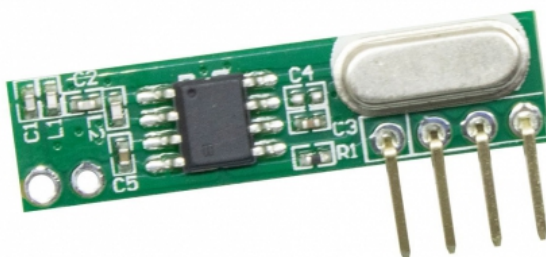
2-7-6- کتابخانه IRremoteESP8266

کتابخانه IRremoteESP8266 یک کتابخانه متن باز برای ارسال و دریافت سیگنال های مادون قرمز روی ESP8266 و ESP32 است. این کتابخانه از گیرنده های IR رایج و تعداد زیادی از پروتکل های کنترل از راه دور پشتیبانی می کند. [6]

2-8-2- ارتباط رادیویی RF



شکل (2-8) گیرنده رادیویی



شکل (2-9) فرستنده رادیویی

2-8-2- مفهوم RF

RF یا Radio Frequency به محدوده ای از فرکانس های امواج الکترومغناطیسی گفته می شود که برای انتقال اطلاعات به صورت بی سیم مورد استفاده قرار می گیرند. سیستم های رادیویی ساده می توانند از یک فرستنده و یک گیرنده تشکیل شوند. بیشتر سیستم های رادیویی از باند های ۳۱۵ و ۴۳۳ مگاهرتز استفاده می کنند.

2-8-3- کتابخانه RCSwitch

کتابخانه RCSwitch برای کنترل بسیاری از تجهیزات رادیویی ساده در بلند های ۳۱۵ و ۴۳۳ مگاهرتز طراحی شده است.

این کتابخانه امکان دریافت و ارسال کد های RF را فراهم کرده و از چیپ ها و پروتکل های رایج تجهیزات RF پشتیبانی می کند. [5]

2-9-2- Wi-Fi ارتباط

2-9-1- معرفی Wi-Fi

Wi-Fi فناوری ارتباط بی سیم مبتنی بر استاندارد های IEEE 802.11 است که امکان اتصال تجهیزات به شبکه را فراهم می کند. ESP32 از Wi-Fi با استاندارد های 802.11b/g/n پشتیبانی می کند. [3]

2-9-2- حالت Station

در حالت Station یا STA، ESP32 مانند یک Client به Access Point متصل می شود. [9]

2-9-3- حالت Access Point

در حالت Access Point یا AP، خود ESP32 یک شبکه Wi-Fi ایجاد می کند و سایر تجهیزات می توانند به آن متصل شوند. [9]

2-9-4- حالت AP_STA

در حالت AP_STA، ESP32 از طریق STA به شبکه اصلی متصل می شود و هم زمان یک Access Point ایجاد می کند که قابلیت متصل شدن به ESP32 بدون قطع شدن از شبکه را فراهم می کند. [9]

2-9-5- Captive Portal

Captive Portal روشی است که در آن دستگاه متصل شده به شبکه، قبل از دسترسی معمول به شبکه با یک صفحه تنظیمات مواجه می شود.

2-10-1- پروتکل MQTT

2-10-1-1 معرفی MQTT

MQTT یک پروتکل پیام رسانی سبک بر پایه مدل Client/Server و Publish/Subscribe است که برای محیط هایی مانند IoT و شبکه های دارای محدودیت منابع طراحی شده است. استاندارد MQTT 5.0 توسط OASIS منتشر شده است. [4]

ویژگی های مهم MQTT عبارتند از:

- سربار کم
- معماری Publish/Subscribe
- جداسازی فرستنده و گیرنده
- پشتیبانی از QoS
- مناسب برای دستگاه های دارای محدودیت
- ارتباط بر بستر TCP/IP

2-10-2- معماری Publish/Subscribe

در MQTT فرستنده و گیرنده مستقیماً با یکدیگر ارتباط ندارند. در عوض یک Broker در میان آن ها قرار دارد.

$Publisher \xrightarrow{Publish} Broker \xrightarrow{Publish} Subscriber$

2-10-3- MQTT Broker

Broker وظیفه دریافت پیام ها و ارسال آن ها به Client هایی را دارد که Topic مربوطه را Subscribe کردند.

2-10-4- Publisher

Client ای که یک پیام در یک Topic منتشر می کند Publisher نام دارد.

Subscriber -2-10-5

Client ای که علاقه مند به دریافت پیام های یک Topic است، آن Topic را Subscribe می کند.

Topic/Payload -2-10-6

Topic یک مسیر منطقی برای دسته بندی پیام های MQTT است.
ساختار کلی Topic ها به صورت زیر است:
`< topic > / < subtopic > / < subtopic >`

Payload محتوای پیام MQTT است.
استاندارد MQTT محتوا و قالب Payload را به کاربرد واگذار می کند؛ بنابراین می توان Payload را به صورت متن، JSON یا داده های دیگر طراحی کرد. [4]

QoS -2-10-7

MQTT سه سطح Quality of Service دارد: [4]

QoS 0:

پیام حداکثر یک بار ارسال می شود و تضمینی برای تحویل وجود ندارد.

QoS 1:

پیام حداقل یک بار ارسال می شود، بنابراین امکان دریافت پیام تکراری وجود دارد.

QoS 2:

پیام تضمین می شود که یک بار تحویل می شود.

WebSocket از طریق MQTT -2-10-8

مرورگر های وب به صورت مستقیم برای ارتباط با MQTT روی TCP مناسب نیستند؛ بنابراین برای اتصال مستقیم مرورگر به MQTT Broker از WebSocket استفاده می کنیم.
WebSocket یک ارتباط دو طرفه و پایدار بین Browser و Server ایجاد می کند.

11-2- ذخیره سازی اطلاعات در ESP32

1-11-2 حافظه Flash

ESP32 دارای حافظه Flash برای نگهداری Firmware و داده های دائمی است. از این حافظه می توان برای اطلاعاتی استفاده کرد که باید بعد از restart نیز باقی بماند.

2-11-2 LittleFS

LittleFS یک فایل سیستم مناسب برای حافظه Flash سیستم های نهفته است. در ESP32، Arduino-ESP32، LittleFS به عنوان یکی از فایل سیستم های قابل استفاده در ESP32 معرفی شده است. [10]

فصل 3:

تحليل و طراحی سیستم

1-3- مقدمه

در این فصل، فرایند تحلیل و طراحی سامانه هوشمند پایش محیط و کنترل تجهیزات مبتنی بر اینترنت اشیا ارائه می شود. هدف از طراحی این سامانه، ایجاد یک هاب هوشمند مبتنی بر میکروکنترلر ESP32 است که بتواند ضمن دریافت و پردازش اطلاعات محیطی، امکان ارسال داده ها از طریق شبکه بی سیم، ایجاد هشدار در شرایط مشخص و همچنین دریافت و بازپخش سیگنال های کنترلی مادون قرمز و فرکانس رادیویی را فراهم کند.

سامانه طراحی شده از مجموعه ای از حسگر های محیطی شامل حسگر دما و رطوبت DHT11، حسگر دود و گاز MQ-2، حسگر نور مبتنی بر LDR و حسگر تشخیص حرکت PIR تشکیل شده است. داده های حاصل از این حسگر ها توسط ESP32 دریافت و پردازش شده و در بازه های زمانی مشخص از طریق شبکه Wi-Fi به MQTT Broker ارسال می شوند.

علاوه بر قابلیت پایش محیط، سامانه دارای زیرسیستم ارتباطی RF در دو باند فرکانسی ۳۱۵ و ۴۳۳ مگاهرتز و یک زیرسیستم ارتباط مادون قرمز است. این زیرسیستم ها امکان دریافت، ذخیره سازی و بازپخش برخی سیگنال های کنترلی تجهیزات مختلف را فراهم می کنند. به این ترتیب، ESP32 به عنوان یک واسط میان تجهیزات فیزیکی، شبکه و رابط کاربری عمل می کند.

2-3- تحلیل نیازمندی های سیستم

با توجه به اهداف پروژه و قابلیت های موردنظر، نیازمندی های سامانه به دو گروه نیازمندی های عملکردی و غیر عملکردی تقسیم شده اند.

1-2-3- نیازمندی های عملکردی

نیازمندی های عملکردی، بیان کننده قابلیت ها و وظایفی هستند که سامانه باید انجام دهد.

مهم ترین نیازمندی های عملکردی سیستم طراحی شده عبارت اند از:

- سامانه باید امکان اندازه گیری دمای محیط را فراهم کند.
- سامانه باید امکان اندازه گیری رطوبت نسبی محیط را فراهم کند.
- سامانه باید بتواند شدت نور محیط را اندازه گیری کند.
- سامانه باید بتواند تغییرات ناشی از وجود دود یا گاز در محیط را شناسایی کند.
- سامانه باید قابلیت تشخیص حرکت در محیط را داشته باشد.
- سامانه باید در صورت عبور مقدار گاز از آستانه تعیین شده، وضعیت هشدار را فعال کند.
- سامانه باید در شرایط هشدار قادر به فعالسازی بازار باشد.
- سامانه باید بتواند وضعیت و اطلاعات حسگرها را از طریق شبکه Wi-Fi منتقل کند.

- سامانه باید قابلیت برقراری ارتباط با یک MQTT Broker را داشته باشد.
- سامانه باید بتواند اطلاعات حسگرها را در Topic های مشخص MQTT منتشر کند.
- سامانه باید بتواند فرمان های دریافت شده از طریق MQTT را پردازش و اجرا کند.
- سامانه باید قابلیت دریافت سیگنال های RF در باند ۳۱۵ مگاهرتز را داشته باشد.
- سامانه باید قابلیت دریافت سیگنال های RF در باند ۴۳۳ مگاهرتز را داشته باشد.
- سامانه باید قابلیت دریافت سیگنال های IR را داشته باشد.
- سامانه باید قابلیت ذخیره سازی سیگنال های RF و IR را فراهم کند.
- سامانه باید قابلیت ارسال مجدد سیگنال های RF و IR ذخیره شده را داشته باشد.
- سامانه باید بتواند سیگنال های ذخیره شده را پس از راه اندازی مجدد بازیابی کند.
- سامانه باید امکان تنظیم اطلاعات شبکه Wi-Fi و MQTT را بدون نیاز به تغییر و کامپایل مجدد Firmware فراهم کند.
- سامانه باید اطلاعات حسگرها و وضعیت هشدار را در یک داشبورد تحت وب نمایش دهد.
- سامانه باید امکان ارسال فرمان های کنترلی از طریق داشبورد تحت وب را داشته باشد.

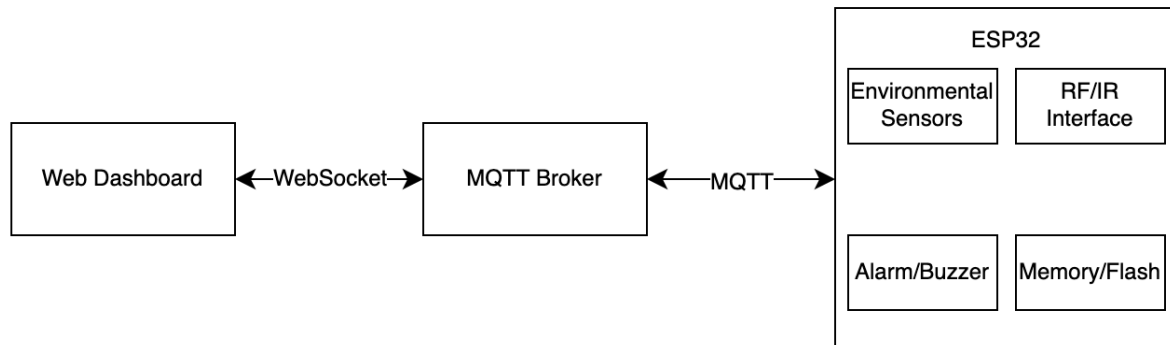
3-2-2- نیازمندی های غیر عملکردی

- نیازمندی های غیر عملکردی ویژگی هایی را مشخص می کنند که کیفیت سامانه را تعیین می کنند. مهم ترین نیازمندی های غیر عملکردی سیستم عبارتند از:
- قابلیت توسعه
 - ساختار Firmware سامانه به صورت ماژولار طراحی شده است تا اضافه کردن حسگرها، تجهیزات یا قابلیت های جدید با حداقل تغییر در سایر بخش های Firmware امکان پذیر باشد.
 - قابلیت نگهداری
 - هر بخش از سیستم، از جمله مدیریت سنسورها، ارتباط MQTT، ارتباط RF، ارتباط IR، مدیریت هشدار و مدیریت حافظه در قالب ماژول های مستقل پیاده سازی شده است. این ساختار موجب ساده تر شدن فرایند بررسی، اصلاح و توسعه نرم افزار می شود.
 - قابلیت پیکربندی
 - اطلاعات مربوط به شبکه Wi-Fi و MQTT نباید به صورت ثابت و غیر قابل تغییر در Firmware قرار داشته باشند. به همین منظور، سامانه دارای یک پورتال پیکربندی است که امکان تغییر تنظیمات شبکه و MQTT را فراهم می کند.
 - پایداری
 - قطع ارتباط با MQTT Broker نباید موجب توقف عملکرد بخش های محلی سامانه مانند خواندن حسگرها، تشخیص حرکت یا عملکرد سیستم هشدار شود. به همین دلیل، ارتباط MQTT به صورت مستقل از سایر فرایندهای اصلی سیستم مدیریت می شود.
 - حفظ اطلاعات
 - تنظیمات شبکه و سیگنال های ذخیره شده RF و IR باید پس از خاموش و روشن شدن دستگاه

نیز حفظ شوند. برای این منظور از سیستم فایل LittleFS موجود در ESP32 استفاده شده است.

- قابلیت دسترسی سامانه باید امکان دسترسی کاربر به اطلاعات محیطی و ارسال فرمان های کنترلی را از طریق رابط وب فراهم کند. ارتباط داشبورد با سامانه از طریق MQTT و WebSocket انجام می شود.

3-3- معماری کلی سیستم



شکل (3-1) معماری کلی سیستم

معماری سامانه به گونه ای طراحی شده است که ESP32 به عنوان هسته پردازشی و ارتباطی، در مرکز تعامل میان حسگرها، تجهیزات کنترلی، شبکه و کاربر قرار می گیرد. در این معماری، اطلاعات محیطی توسط حسگر های متصل به ESP32 دریافت شده و پس از پردازش، از طریق شبکه Wi-Fi و پروتکل MQTT به Broker ارسال می شوند.

در سمت دیگر، کاربر می تواند از طریق داشبورد تحت وب به اطلاعات سامانه دسترسی داشته و فرمان های کنترلی موردنظر خود را ارسال کند. فرمان ها پس از عبور از MQTT Broker به ESP32 منتقل شده و توسط بخش مدیریت فرمان پردازش می شوند. معماری کلی سیستم را می توان به سه بخش اصلی تقسیم کرد:

- لایه سخت افزار و جمع آوری داده

- لایه پردازش و کنترل مبتنی بر ESP32

- لایه ارتباطی و رابط کاربر

نقشه شماتیک مدار را می توانید در بخش 1-8- مشاهده کنید.

3-3-2- لایه سخت افزار و جمع آوری داده

در لایه سخت افزاری، مجموعه ای از حسگرها و ماژول های ارتباطی به برد ESP32 متصل شده اند. این اجزا وظیفه دریافت اطلاعات محیطی و ارتباط با تجهیزات خارجی را بر عهده دارند. حسگر های مورد استفاده در این لایه عبارت اند از:

- DHT11 برای اندازه گیری دما و رطوبت
- MQ-2 برای اندازه گیری سطح گاز و رطوبت
- LDR برای اندازه گیری شدت نور محیط
- PIR برای تشخیص حرکت

همچنین تجهیزات ارتباطی شامل گیرنده و فرستنده RF در فرکانس های ۳۱۵ و ۴۳۳ مگاهرتز و گیرنده و فرستنده مادون قرمز هستند.

ESP32 داده های دریافتی از این تجهیزات را دریافت و برای پردازش در اختیار ماژول های نرم افزاری مربوطه قرار می دهد.

3-3-3- لایه پردازش و کنترل

ESP32 هسته اصلی پردازش سامانه است. این بخش مسئولیت دریافت داده های حسگرها پردازش اطلاعات، مدیریت وضعیت هشدار، کنترل تجهیزات RF و IR، مدیریت حافظه و برقراری ارتباط با MQTT Broker را بر عهده دارد.

نرم افزار Firmware به صورت ماژولار طراحی شده است. در این ساختار، وظایف مختلف سیستم میان ماژول های مستقل تقسیم شده اند. برای نمونه، ماژول های اختصاصی برای سنسور های MQ-2, LDR, PIR, DHT11 در نظر گرفته شده و مدیریت MQTT, IR, RF و حافظه نیز در ماژول های جداگانه انجام می شود.

این معماری باعث می شود تغییر یا توسعه یک بخش، وابستگی کمتری به سایر بخش های سیستم داشته باشد.

3-3-4- لایه ارتباطی و رابط کاربر

ارتباط میان ESP32 و زیرساخت نرم افزاری سامانه از طریق Wi-Fi برقرار می شود. پس از اتصال ESP32 به شبکه، داده های اندازه گیری شده توسط پروتکل MQTT به Broker ارسال می شوند. در این پروژه از broker.emqx.io به عنوان MQTT Broker استفاده شده است. داشبورد تحت وب نیز از طریق WebSocket به MQTT Broker متصل می شود. به این ترتیب، داشبورد قادر است داده های منتشر شده توسط ESP32 را دریافت کرده و وضعیت فعلی سیستم را به کاربر نمایش دهد و فرمان های ارسال شده توسط کاربر نیز از طریق همین زیرساخت به ESP32 منتقل می شوند.

3-4- طراحی سخت افزار

در طراحی سخت افزاری سامانه از ماژول ESP32-WROOM-32 به عنوان واحد پردازشی اصلی استفاده شده است. انتخاب این ماژول به دلیل برخورداری از قابلیت های پردازشی مناسب، تعداد کافی پایه های

ورودی و خروجی، مبدل ADC و همچنین پشتیبانی داخلی از ارتباط Wi-Fi صورت گرفته است.

1-4-3- واحد پردازش مرکزی

ماژول ESP32-WROOM-32 به عنوان کنترل کننده اصلی سامانه انتخاب شده است. این برد وظیفه دریافت اطلاعات از حسگرها، پردازش داده ها، مدیریت تجهیزات ارتباطی، اجرای منطق هشدار، ذخیره سازی اطلاعات و برقراری ارتباط با شبکه را بر عهده دارد. از قابلیت Wi-Fi داخلی ESP32 برای اتصال سامانه به شبکه محلی و برقراری ارتباط با MQTT Broker استفاده شده است. همچنین تعدادی از پایه های GPIO برای اتصال تجهیزات دیجیتال و پایه های ADC برای دریافت خروجی های آنالوگ حسگر های محیطی مورد استفاده قرار گرفته است.

2-4-3- سیستم اندازه گیری دما و رطوبت

برای اندازه گیری دمای محیط و رطوبت نسبی از حسگر DHT11 استفاده شده است. این حسگر از طریق یک خط داده دیجیتال با ESP32 ارتباط برقرار می کند. برای خواندن اطلاعات از حسگر از کتابخانه Adafruit DHT و توابع readTemperature() و readHumidity() استفاده شده است و پایه داده DHT11 به GPIO15 متصل شده است. اطلاعات دریافت شده از حسگر توسط Firmware پردازش شده و مقادیر دما و رطوبت به صورت جداگانه در سامانه ثبت و از طریق MQTT منتشر می شوند.

3-4-3- سیستم تشخیص دود و گاز

برای پایش وجود دود و برخی گاز های قابل اشتعال از حسگر MQ-2 استفاده شده است. در این پروژه خروجی آنالوگ حسگر مورد استفاده قرار گرفته و به پایه GPIO33 از ESP32 متصل شده است. حسگر MQ-2 برای کار رسیدن به وضعیت کاری مطلوب نیاز به پیش گرمایش دارد که از طریق GPIO26 فرمان گرم شدن به حسگر داده می شود. طبق اطلاعات فنی ارائه شده برای راه اندازی اولیه، سنسور باید در حدود ۲۴ ساعت گرم شود و سپس امکان خواندن مقدار حسگر وجود دارد. مقدار خوانده شده از خروجی آنالوگ MQ-2 توسط مبدل آنالوگ به دیجیتال ESP32 دریافت می شود. Firmware مقدار حاصل را از طریق MQTT به داشبورد ارسال کرده و همچنین با یک آستانه از پیش تعیین شده مقایسه می کند. در صورت عبور مقدار اندازه گیری شده از آستانه تعیین شده، وضعیت هشدار را فعال کرده و بازر را به صدا در می آورد و وضعیت هشدار را از طریق MQTT به داشبورد ارسال می کند.

3-4-4- سیستم اندازه گیری شدت نور

برای اندازه گیری میزان نور محیط از LDR استفاده شده است. خروجی آنالوگ این ماژول به GPIO32 متصل شده و مقدار آن توسط ADC خوانده می شود. مقدار حاصل از LDR در بازه های زمانی مشخص توسط Firmware نمونه برداری شده و از طریق MQTT منتشر می شود.

3-4-5- سیستم تشخیص حرکت

برای تشخیص حرکت از ماژول PIR استفاده شده است. خروجی دیجیتال این ماژول به GPIO13 متصل شده است. در صورت تشخیص حرکت، وضعیت خروجی PIR تغییر کرده و Firmware این تغییر را پردازش می کند و از طریق MQTT منتشر می کند. در صورت فعال بودن قابلیت تشخیص حرکت، وقوع حرکت می تواند موجب فعال شدن سیستم هشدار و ارسال پیام مربوطه از طریق MQTT به داشبورد شود.

3-4-6- سیستم ارتباط فرکانس رادیویی

یکی از قابلیت های اصلی سامانه، دریافت و بازپخش سیگنال های رادیویی در دو باند فرکانسی ۳۱۵ و ۴۳۳ مگاهرتز است. به دلیل تفاوت فرکانس این دو گروه، برای هر باند یک گیرنده و فرستنده مستقل در نظر گرفته شده است. این ساختار به ESP32 این امکان را می دهد که سیگنال های دو باند را به صورت مستقل دریافت و ارسال کند. برای باند ۳۱۵ مگاهرتز از ماژول گیرنده RXB61 و ماژول فرستنده H34S استفاده شده است که خروجی دیتا گیرنده به GPIO22 و ورودی دیتا فرستنده به GPIO23 متصل شده است. برای باند ۴۳۳ مگاهرتز از ماژول گیرنده RXB35 و ماژول فرستنده H34S استفاده شده است که خروجی دیتا گیرنده به GPIO14 و ورودی دیتا فرستنده به GPIO27 متصل شده است. در فرایند گرفتن سیگنال، ESP32 با هر دو گیرنده RF ارتباط برقرار می کند و گیرنده، سیگنال ارسال شده از ریموت های RF را دریافت کرده و اطلاعات آن توسط ESP32 پردازش می شود. سیگنال های دریافت شده با نام مشخص در حافظه ذخیره می شوند تا در زمان دیگری مورد استفاده قرار گیرند. در فرایند ارسال سیگنال نیز ESP32 بر اساس باند فرکانسی سیگنال مورد ارسال، فرستنده مناسب را انتخاب کرده و اطلاعات سیگنال را به فرستنده می فرستد. برای ارتباط برقرار کردن با ماژول ها از کتابخانه RCSSwitch استفاده شده است.

3-4-7- سیستم ارتباط مادون قرمز

برای ارتباط مادون قرمز از یک گیرنده مادون قرمز و چهار LED مادون قرمز استفاده شده است. گیرنده مادون قرمز به GPIO17 و LED های مادون قرمز به GPIO4 متصل شده است. گیرنده IR، سیگنال های ارسال شده توسط کنترل های مادون قرمز را دریافت کرده و اطلاعات آن ها توسط ESP32 پردازش می شود. سیگنال های دریافت شده با نام مشخص در سیستم ذخیره می شود تا در زمان دیگری مورد استفاده قرار گیرند. برای ارسال سیگنال، ESP32 اطلاعات ذخیره شده در سیگنال را بازیابی کرده و از طریق LED های مادون قرمز، آن را بازپخش می کند. برای دریافت و ارسال سیگنال ها از کتابخانه IRremoteESP8266 استفاده شده است.

3-4-8- سیستم هشدار صوتی

برای ایجاد هشدار صوتی از یک Active Buzzer استفاده شده است. ورودی این ماژول به GPIO19 متصل شده است. بازر در شرایطی مانند عبور مقدار MQ-2 از آستانه تعیین شده یا تشخیص حرکت در حالت فعال، می تواند توسط Firmware فعال شود.

3-4-9- LED وضعیت

یک LED وضعیت برای ارائه بازخورد بصری از وضعیت عملکرد سیستم در نظر گرفته شده است. این LED در Firmware برای نمایش وضعیت فعالیت دستگاه و همچنین وقوع برخی رویداد های مرتبط با دریافت یا ارسال سیگنال استفاده می شود. این LED به GPIO25 متصل شده است.

3-5- پایه های ESP32

جدول زیر ارتباط تجهیزات اصلی با پایه های ESP32 را نشان می دهد:

جدول (3-1) پایه های ESP32

ردیف	تجهیز	پایه ESP32	نوع ورودی/خروجی
۱	DHT11	GPIO15	ورودی دیجیتال
۲	MQ-2 Heater	GPIO26	خروجی دیجیتال
۳	MQ-2 ADC	GPIO33	ورودی آنالوگ
۴	LDR ADC	GPIO32	ورودی آنالوگ

ورودی دیجیتال	GPIO13	PIR	۵
ورودی دیجیتال	GPIO22	RF 315Mhz Receiver	۶
خروجی دیجیتال	GPIO23	RF 315Mhz Transmitter	۷
ورودی دیجیتال	GPIO14	RF 433Mhz Receiver	۸
خروجی دیجیتال	GPIO27	RF 433Mhz Transmitter	۹
ورودی دیجیتال	GPIO17	IR Receiver	۱۰
خروجی دیجیتال	GPIO4	IR LED	۱۱
خروجی دیجیتال	GPIO19	Buzzer	۱۲
خروجی دیجیتال	GPIO25	Status LED	۱۳

3-6- طراحی نرم افزار و ساختار Firmware

Firmware بر روی ماژول ESP32-WROOM-32 و با استفاده از محیط توسعه PlatformIO و چارچوب Arduino پیاده سازی شده است. با توجه به تعدد وظایف سیستم، Firmware به صورت ماژولار طراحی شده است تا هر بخش از سامانه مسئولیت مشخصی داشته باشد و وابستگی میان اجزای مختلف کاهش یابد.

3-6-1 ساختار ماژولار Firmware

جدول زیر وظیفه هر ماژول را نشان می دهد:

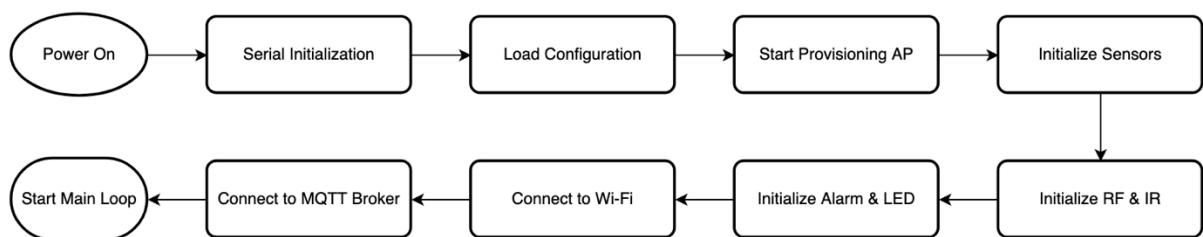
جدول (3-2) وظیفه ماژول های Firmware

وظیفه	ماژول
راه اندازی و کنترل جریان اصلی برنامه	main
مدیریت دوره ای حسگرها و رویداد های محیطی	Sensors
دریافت دما و رطوبت	DHT
دریافت مقدار گاز و دود	MQ2
دریافت مقدار نور	LDR
تشخیص حرکت	PIR
برقراری ارتباط با Broker و ارسال و دریافت پیام	MQTT
دریافت و ارسال سیگنال های RF	RF
دریافت و ارسال سیگنال های مادون قرمز	IR
مدیریت وضعیت هشدار و بازر	Alarm
مدیریت LED وضعیت	LED
پیکربندی Wi-Fi و MQTT	Provisioning

ذخیره و بازیابی اطلاعات در LittleFS	Memory
چاپ و دریافت و پردازش فرمان های متنی	Serial
نگهداری تنظیمات و پارامتر های ثابت	Config

3-6-2- فرایند راه اندازی سیستم

پس از روشن شدن ESP32، تابع `setup()` اجرا می شود. این تابع وظیفه آماده سازی اولیه اجزای سیستم را بر عهده دارد. روند راه اندازی سیستم به صورت زیر است:

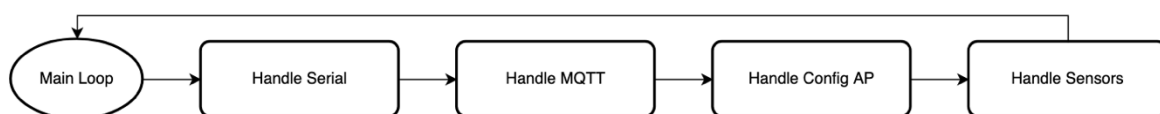


شکل (3-2) روند راه اندازی سیستم

در ابتدا ارتباط سریال با کامپیوتر برقرار می شود و سپس تنظیمات و پارامتر ها خوانده می شود و سپس اگر نیاز باشد پورتال برای گرفتن اطلاعات AP و MQTT Broker باز می شود. پس از تکمیل شدن اطلاعات، حسگرها و ماژول های ارتباطی راه اندازی می شود و سپس به شبکه Wi-Fi متصل شده و سپس به MQTT Broker متصل می شود.

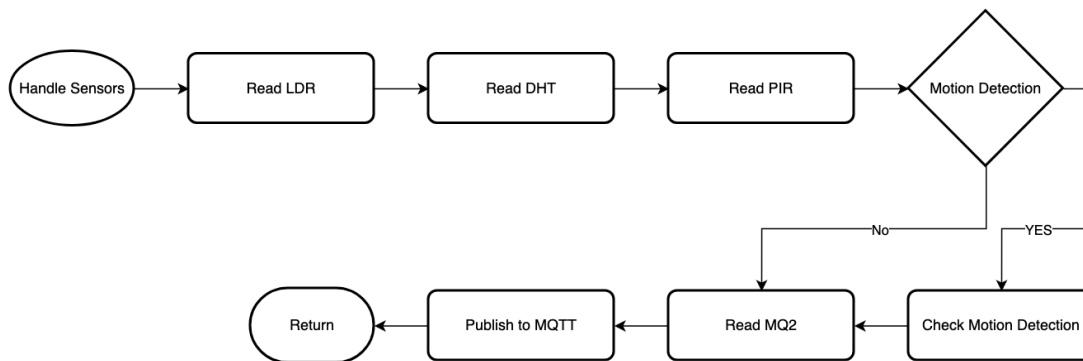
3-6-3- حلقه اصلی اجرای برنامه

پس از پایان عملیات راه اندازی، برنامه وارد تابع `loop()` می شود. این تابع به صورت مداوم اجرا شده و وظایف اصلی سامانه را مدیریت می کند. در هر بار اجرا، سریال ترمینال و MQTT برای دستور های جدید خوانده می شود و سپس حسگرها مدیریت می شوند. روند حلقه اصلی سیستم به صورت زیر است:



شکل (3-3) روند حلقه اصلی سیستم

3-6-4- مدیریت حسگرها

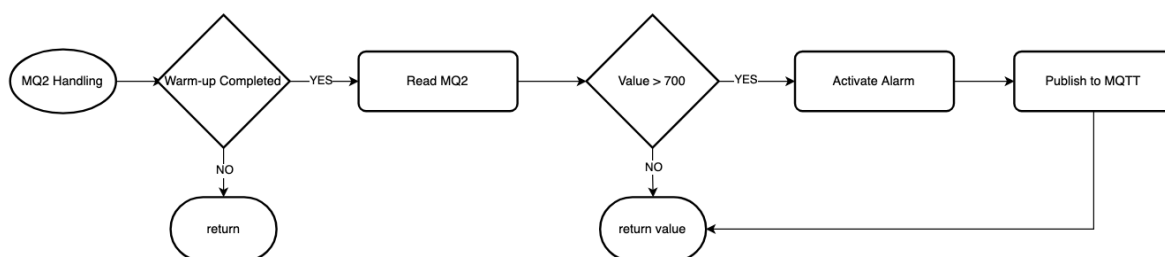


شکل (3-4) روند مدیریت حسگرها

مدیریت حسگرها در ماژول Sensors انجام می شود. این ماژول وظیفه هماهنگی میان درایورهای مختلف حسگرها را بر عهده دارد.

اطلاعات حسگرهای PIR، LDR، DHT11 و MQ-2 در فواصل زمانی مشخص دریافت می شوند. پس از دریافت داده ها، مقادیر دریافت شده در Topic های مربوطه توسط MQTT منتشر می شوند. در نسخه فعلی، بازه زمانی برای دریافت اطلاعات حسگرها برابر با ۱۰ ثانیه است.

3-6-5- مدیریت حسگر MQ-2



شکل (3-5) روند مدیریت حسگر MQ-2

با توجه به ماهیت حسگر MQ-2، این حسگر بلافاصله پس از روشن شدن سیستم، برای استفاده در نظر گرفته نمی شود. Firmware یک دوره گرم شدن اولیه برای حسگر در نظر گرفته است.

پس از پایان این دوره، مقدار خروجی حسگر در بازه های مشخص زمانی توسط ماژول Sensors خوانده می شود و با مقدار آستانه تعیین شده مقایسه می شود. در صورت عبور مقدار خوانده شده از آستانه، سیستم، وضعیت هشدار را فعال کرده و پیام مربوطه را از طریق MQTT در Topic مشخص شده منتشر می کند. در نسخه فعلی، گرم شدن اولیه به دلیل کمبود زمان، ۵ دقیقه در نظر گرفته شده است و همچنین آستانه برابر با ۷۰۰ در نظر گرفته شده است.

3-6-6- مدیریت حسگر DHT11

دما و رطوبت محیط توسط حسگر DHT11 اندازه گیری می شود. مقدار خروجی حسگر در بازه های زمانی مشخص توسط ماژول Sensors خوانده می شود. برای استفاده از این حسگر، از کتابخانه Adafruit DHT استفاده شده است. در Firmware برای اندازه گیری دما از تابع readTemperature و برای اندازه گیری رطوبت از تابع readHumidity استفاده شده است.

3-6-7- مدیریت حسگر PIR

تشخیص حرکت توسط حسگر PIR انجام می شود. وضعیت خروجی این حسگر توسط ESP32 بررسی می شود.

سیستم، علاوه بر دریافت وضعیت PIR، امکان فعال یا غیرفعال کردن قابلیت تشخیص حرکت را نیز فراهم می کند. برای این منظور از فرمان های زیر استفاده شده است:

motion on

motion off

در صورت فعال بودن قابلیت تشخیص حرکت و شناسایی حرکت، سیستم وضعیت هشدار را فعال کرده و پیام مربوطه را در Topic مشخص شده منتشر می کند.

3-6-8- مدیریت سیستم هشدار

ماژول Alarm وظیفه مدیریت وضعیت هشدار و کنترل Buzzer را بر عهده دارد. دو رویداد اصلی می توانند باعث ایجاد وضعیت هشدار شوند:

- عبور مقدار MQ-2 از آستانه تعیین شده
 - تشخیص حرکت در حالتی که تشخیص حرکت فعال باشد
- همچنین امکان کنترل مستقیم بازر از طریق فرمان های سیستم وجود دارد:

buzzer on

buzzer off

در هنگام وقوع هشدار، بازر مطابق منطق زمانی تعریف شده، فعال و غیرفعال می شود. در نسخه فعلی، چرخه زمانی برابر با ۲ ثانیه در نظر گرفته شده است.

3-7- طراحی ارتباط Wi-Fi و MQTT

3-7-1- Wi-Fi ارتباط

ارتباط شبکه ای سامانه از طریق Wi-Fi داخلی ESP32 برقرار می شود. Wi-Fi وظیفه اتصال دستگاه به شبکه و فراهم کردن مسیر ارتباطی میان ESP32 و MQTT Broker را بر عهده دارد. سیستم از حالت WIFI_AP_STA استفاده می کند. در این حالت، ESP32 می تواند به صورت همزمان:

- به شبکه Wi-Fi موردنظر به عنوان Station متصل شود
 - یک Access Point اختصاصی برای پیکربندی ایجاد کند
- این روش امکان پیکربندی مجدد دستگاه را بدون نیاز به تغییر Firmware فراهم می کند.

3-7-2- MQTT ارتباط

برای تبادل داده میان ESP32 و Broker و داشبورد از پروتکل MQTT استفاده شده است. ارتباط MQTT از مدل Publisher/Subscriber استفاده می کند. در این مدل، ESP32 اطلاعات حسگرها را در Topic های مشخص منتشر می کند و در عین حال می تواند پیام های کنترلی منتشر شده توسط داشبورد را دریافت کند. در این پروژه MQTT Broker مورد استفاده broker.emqx.io است.

3-7-2-1- ساختار Topic های MQTT

ساختار Topic ها به صورت زیر طراحی شده است:

$\langle base \rangle / \langle device \rangle / \langle subtopic \rangle$

در این پروژه، مقدار پایه برابر homeintelligence قرار داده شده است و نام دستگاه توسط کاربر تعیین می شود.

3-7-2-2- Topic های مربوط به حسگرها

داده های حسگرها در Topic های مجزا منتشر می شود.

جدول (3-3) Topic های مربوط به حسگرها

داده	Topic
دما	.../sensor/TEMP
رطوبت	.../sensor/HUM
شدت نور	.../sensor/LDR
تشخیص حرکت	.../sensor/PIR

دود و گاز	.../sensor/MQ2
-----------	----------------

3-7-2-3 Topic هشدار

برای اطلاع رسانی رویداد های هشدار از Topic: .../alarm استفاده می شود. پیام های قابل انتشار در این Topic شامل موارد زیر است:

- MQ2 threshold exceed
- Motion Detected
- Motion ON
- Motion OFF

3-7-2-4 Topic فرمان

فرمان های کنترلی از طریق Topic: .../command دریافت می شود. Firmware پیام های دریافت شده از این Topic را به Command Handler ارسال کرده و عملیات متناظر با فرمان، اجرا می شود.

3-7-2-5 Topic فهرست سیگنال های RF و IR

برای ارسال فهرست سیگنال های RF و IR به داشبورد از Topic های .../RF/list و Topic: .../IR/list استفاده می شود. پیام های فرستاده شده در این Topic در قالب JSON هستند.

3-8 طراحی سیستم RF

یکی از قابلیت های اصلی پروژه، امکان دریافت و بازپخش سیگنال های RF در دو باند فرکانسی ۳۱۵ و ۴۳۳ مگاهرتز است.

از آنجا که گیرنده و فرستنده های RF مورد استفاده تک فرکانس هستند، برای هر باند یک مسیر دریافت و ارسال مستقل در نظر گرفته شده است.

برای مدیریت سیگنال های RF از کتابخانه RCSwitch استفاده شده است.

3-8-1 دریافت سیگنال RF

فرمان زیر برای شروع فرایند دریافت استفاده می شود:

`rfrx <name>`

پس از اجرای فرمان، سیستم هر دو گیرنده RF را بررسی می کند. هنگامی که یک سیگنال دریافت شود، اطلاعات مربوط به آن استخراج شده و باند فرکانسی مورد استفاده نیز مشخص می شود.

3-8-2- ذخیره سیگنال RF

پس از دریافت سیگنال، اطلاعات آن در حافظه دائمی سیستم به صورت JSON ذخیره می شود. فایل مربوط به این اطلاعات rf_signals.json است.

3-8-3- بازپخش سیگنال RF

برای ارسال مجدد یک سیگنال ذخیره شده از فرمان زیر استفاده می شود:

rf tx <name>

Firmware باند فرکانسی ذخیره شده را بررسی کرده و سیگنال را از طریق فرستنده مناسب ارسال می کند. بنابراین کاربر هنگام ارسال مجدد نیازی به تعیین فرکانس ندارد و سیستم به صورت خودکار فرستنده مناسب را انتخاب می کند.

3-8-4- مدیریت سیگنال های ذخیره شده RF

فرمان های زیر برای مدیریت سیگنال های RF ذخیره شده در نظر گرفته شده است:

rf list

rf del <name>

فرمان list، فهرست سیگنال های ذخیره شده را نمایش داده و اطلاعات آن ها را از طریق MQTT نیز منتشر می کند.

فرمان del برای حذف یک سیگنال مشخص از حافظه استفاده می شود.

3-9- طراحی سیستم مادون قرمز

سیستم مادون قرمز از دو بخش اصلی گیرنده و فرستنده تشکیل شده است. برای مدیریت سیگنال های مادون قرمز از کتابخانه IRremoteESP8266 استفاده شده است.

3-9-1- دریافت سیگنال IR

برای شروع دریافت یک سیگنال IR از فرمان زیر استفاده می شود:

ir rx <name>

پس از اجرای فرمان، سیستم منتظر دریافت یک سیگنال مادون قرمز می ماند. در زمان دریافت یک سیگنال، سیستم اطلاعات مربوط آن را استخراج می کند.

سیستم قادر به پردازش برخی پروتکل های رایج مانند SONY, NEC, SAMSUNG و PANASOINC بوده و در صورت ناشناخته بودن پروتکل، امکان ذخیره اطلاعات خام سیگنال نیز وجود دارد.

3-9-2- ذخیره سیگنال IR

سیگنال های دریافت شده در حافظه دائمی سیستم به صورت JSON ذخیره می شود. فایل مربوط به این اطلاعات ir_signals.json است.

3-9-3- بازپخش سیگنال IR

برای بازپخش سیگنال ذخیره شده از فرمان زیر استفاده می شود:

ir tx <name>

در این فرایند، Firmware اطلاعات سیگنال را بازپایی کرده و آن را توسط LED مادون قرمز ارسال می کند. در صورتی که پروتکل سیگنال پشتیبانی شود، داده توسط پروتکل فرستاده می شود در غیر اینصورت داده خام ذخیره شده به صورت سیگنال فرستاده می شود.

3-9-4- مدیریت سیگنال های ذخیره شده IR

فرمان های زیر برای مدیریت سیگنال های IR ذخیره شده در نظر گرفته شده است:

ir list

ir del <name>

فرمان list، فهرست سیگنال های ذخیره شده را نمایش داده و اطلاعات آن ها را از طریق MQTT نیز منتشر می کند.

فرمان del برای حذف یک سیگنال مشخص از حافظه استفاده می شود.

3-10- طراحی سیستم ذخیره سازی

برای ذخیره سازی اطلاعات دائمی از فایل سیستم LittleFS استفاده شده است. استفاده از سیستم فایل داخلی ESP32 امکان نگهداری اطلاعات بدون نیاز به حافظه خارجی را فراهم می کند. در پروژه سه فایل اصلی ایجاد می شوند:

جدول (3-4) فایل های اطلاعات دائمی سیستم

فایل	محتوا
------	-------

تنظیمات Wi-Fi و MQTT	/provisioning.json
سیگنال های ذخیره شده RF	/rf_signals.json
سیگنال های ذخیره شده IR	/ir_signals.json
تنظیمات دستگاه	/settings.json

اطلاعات این فایل ها پس از راه اندازی مجدد دستگاه قابل بازیابی هستند.

3-10-2- ذخیره تنظیمات Provisioning

اطلاعات مربوط به شبکه و MQTT به صورت JSON در فایل provisioning.json ذخیره می شوند. این اطلاعات شامل موارد زیر است:

- SSID شبکه Wi-Fi
- رمز عبور Wi-Fi
- آدرس MQTT Broker
- پورت MQTT
- نام کاربری و رمز عبور MQTT در صورت استفاده
- نام اختصاصی دستگاه

3-10-3- ذخیره تنظیمات دستگاه

اطلاعات مربوط به تنظیمات دستگاه که با خاموش شدن دستگاه باید ذخیره شوند، به صورت JSON در فایل settings.json ذخیره می شوند. این اطلاعات شامل موارد زیر است:

- وضعیت فعال بودن تشخیص حرکت

3-10-4- ساختار ذخیره سیگنال RF

اطلاعات سیگنال های RF به صورت JSON در فایل rf_signals.json ذخیره می شوند. اطلاعات مربوط به هر سیگنال شامل موارد زیر است:

- نام سیگنال
- کد سیگنال
- تعداد بیت
- پروتکل

- طول پالس
- باند فرکانسی

3-10-5- ساختار ذخیره سیگنال IR

اطلاعات سیگنال های IR به صورت JSON در فایل ir_signals.json ذخیره می شوند. اطلاعات مربوط به هر سیگنال شامل موارد زیر است:

- نام سیگنال
- پروتکل
- کد سیگنال
- تعداد بیت
- اطلاعات داده خام در صورت نیاز

3-11- طراحی سیستم Provisioning

یکی از قابلیت های نرم افزاری سامانه، امکان پیکربندی Wi-Fi و MQTT بدون تغییر Firmware است. برای این منظور ESP32 در حالت WIFI_AP_STA فعال می شود. در این حالت، دستگاه علاوه بر تلاش برای اتصال به شبکه Wi-Fi، یک Access Point اختصاصی نیز ایجاد می کند. مشخصات اتصال به AP در جدول زیر آمده است.

جدول (3-5) مشخصات AP

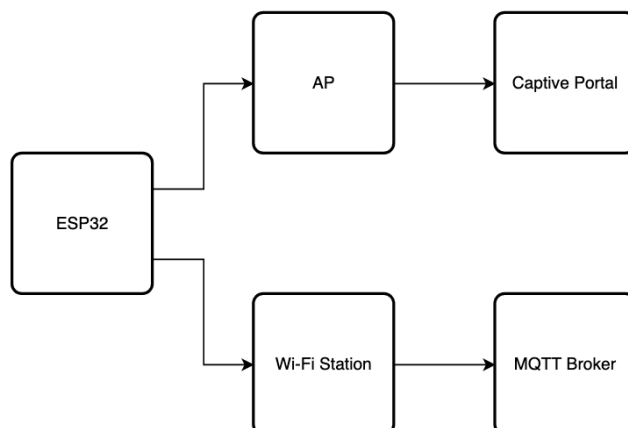
مقدار	مشخصه
Intelligence_Setup	SSID
12345678	Password
192.168.4.1	IP

3-11-2- Captive Portal

پس از اتصال کاربر به Access Point، یک Captive Portal در اختیار وی قرار می گیرد. این صفحه امکان وارد کردن اطلاعات مورد نیاز برای اتصال دستگاه به شبکه و MQTT Broker را فراهم می کند. پس از ذخیره تنظیمات، اطلاعات در LittleFS ذخیره شده و دستگاه Restart می شود.

3-11-3- اتصال همزمان AP و Station

در طراحی سیستم، Access Point مربوط به Provisioning تنها در زمان راه اندازی اولیه نیست، بلکه در حالت عادی نیز فعال باقی می ماند.



شکل (3-6) نمودار اتصال به شبکه

این ویژگی امکان تغییر تنظیمات شبکه یا MQTT را در زمانی که ESP32 به شبکه متصل است، فراهم می کند.

3-12- طراحی Web Dashboard

داشبورد به صورت Server-less طراحی شده و مستقیماً از طریق WebSocket به MQTT متصل می شود. از آنجا که مرورگر به صورت مستقیم از MQTT روی TCP پشتیبانی نمی کند، Broker باید قابلیت WebSocket را فعال کرده باشد.

3-12-1- نمایش اطلاعات حسگرها

داشبورد اطلاعات دریافت شده از Topic های مربوط به حسگرها را دریافت کرده و به کاربر نمایش می دهد. داده ها به صورت زنده از MQTT Broker دریافت می شوند و نیاز به Refresh کردن صفحه برای مشاهده تغییرات وجود ندارد.

3-12-2- نمایش وضعیت هشدار

در صورت دریافت پیام هشدار از Topic مربوطه، داشبورد وضعیت هشدار را به کاربر نمایش می دهد.

3-12-3- ارسال فرمان از طریق داشبورد

داشبورد علاوه بر نمایش اطلاعات، امکان ارسال فرمان به ESP32 را نیز فراهم می کند. این فرمان ها می توانند شامل موارد زیر باشند:

- buzzer on
- buzzer off
- motion on
- motion off
- rf rx <name>
- rf tx <name>
- rf del <name>
- rf list
- ir rx <name>
- ir tx <name>
- ir del <name>
- ir list

3-13- طراحی مدیریت خطا و ارتباطات

با توجه به وابستگی بخش ارتباطی سیستم به شبکه، احتمال قطع Wi-Fi یا در دسترس نبودن MQTT Broker وجود دارد. به همین دلیل، Firmware فرایند اتصال شبکه و اتصال MQTT را به صورت مستقل مدیریت می کند.

در صورت قطع ارتباط با MQTT Broker، سیستم تلاش می کند ارتباط را مجدداً برقرار کند. در این شرایط، فرایندهای محلی دستگاه متوقف نمی شود و قابلیت هایی مانند خواندن حسگرها، مدیریت هشدار و عملکرد بخش RF/IR همچنان توسط Firmware اجرا می شوند.

3-14- کتابخانه های نرم افزاری مورد استفاده

برای پیاده سازی Firmware از چند کتابخانه استفاده شده است که در جدول زیر ذکر شده است.

جدول (3-6) کتابخانه های مورد استفاده

کتابخانه	کاربرد
DHT sensor library	ارتباط با حسگر DHT11
Adafruit Unified sensor	لایه مشترک مورد استفاده کتابخانه های حسگر Adafruit
RCSwitch	دریافت و ارسال سیگنال های RF
IRremoteESP8266	دریافت و ارسال سیگنال های IR
ArduinoJSON	پردازش و ذخیره داده های JSON
PubSubClient	پیاده سازی ارتباط MQTT

فصل 4:

پیاده سازی سامانه

1-4- مقدمه

در این فصل، فرایند پیاده سازی سامانه تشریح می شود. پس از بررسی مبانی نظری و طراحی سامانه در فصل های پیشین، در این فصل نحوه تبدیل طراحی ارائه شده به یک سامانه عملیاتی مورد بررسی قرار می گیرد.

پیاده سازی انجام شده شامل دو بخش اصلی Firmware دستگاه و رابط کاربری تحت وب است. Firmware بر روی برد ESP32 و با استفاده از چارچوب Arduino توسعه داده شده و وظیفه مدیریت حسگرها، پردازش داده های دریافتی، کنترل سیستم هشدار، دریافت و ارسال سیگنال های RF و IR، مدیریت ارتباط Wi-Fi و MQTT و همچنین ذخیره سازی اطلاعات را بر عهده دارد. در بخش سخت افزاری، ESP32 به مجموعه ای از حسگرها و تجهیزات جانبی شامل حسگر دما و رطوبت DHT11، حسگر گاز و دود MQ-2، حسگر نور مبتنی بر LDR، حسگر تشخیص حرکت PIR، گیرنده و فرستنده RF در باند های ۳۱۵ و ۴۳۳ مگاهرتز، گیرنده و فرستنده مادون قرمز و بازر متصل شده است. اطلاعات دریافت شده از حسگر ها توسط ESP32 پردازش شده و در فواصل زمانی مشخص فرستاده می شود.

در بخش ارتباطی دستگاه با استفاده از Wi-Fi به شبکه متصل شده و برای تبادل داده و دریافت فرمان های کنترلی از پروتکل MQTT استفاده می کند. MQTT Broker مورد استفاده در پیاده سازی نهایی پروژه، سرویس عمومی broker.emqx.io است. همچنین برای پیکربندی اولیه و تغییر تنظیمات شبکه، زیرسامانه مبتنی بر Access Point و Captive Portal در Firmware پیاده سازی شده است. یکی دیگر از قابلیت های اصلی سامانه، امکان دریافت، ذخیره و بازپخش سیگنال های RF و IR است. این قابلیت دستگاه را از یک سامانه صرفاً نظارتی به یک واسط ارتباطی میان تجهیزات الکترونیکی و زیرساخت هوشمند تبدیل می کند.

درنهایت، یک Web Dashboard برای مشاهده داده های حسگرها، دریافت هشدار ها و ارسال فرمان های کنترلی طراحی شده است. این داشبورد مستقیماً با MQTT Broker ارتباط برقرار کرده و امکان تعامل کاربر با دستگاه را بدون نیاز به یک Backend اختصاصی فراهم می کند.

2-4- محیط و ابزار های توسعه

تنظیمات مربوط به محیط توسعه و وابستگی های پروژه در فایل platformio.ini تعریف شده است. این فایل به عنوان فایل پیکربندی اصلی محیط ساخت پروژه، مشخصات برد، Framework، سرعت ارتباط سریال، سیستم فایل و کتابخانه های مورد استفاده را تعیین می کند.

4-2-1- Framework تنظیمات برد و

در این پروژه محیط توسعه برای برد ESP32 از پلتفرم espressif32 استفاده می کند. Framework انتخاب شده نیز Arduino است.

پارامتر platform مشخص کننده پلتفرم توسعه مربوط به خانواده ESP32 است. پارامتر board نیز پیکربندی سخت افزاری مربوط به برد مورد استفاده را مشخص می کند. در این پروژه از تنظیمات عمومی esp32dev استفاده شده است.

4-2-2- تنظیمات ارتباط سریال

برای مشاهده پیام های خروجی Firmware و بررسی وضعیت عملکرد دستگاه، سرعت ارتباط Serial Monitor برابر با 115200 bauds تعیین شده است.

این ارتباط در فرایند توسعه و آزمایش سامانه کاربرد دارد و امکان مشاهده پیام های مربوط به راه اندازی دستگاه، وضعیت اتصال شبکه، اطلاعات حسگرها و دریافت و ارسال فرمان ها در Firmware را فراهم می کند.

4-2-3- سیستم فایل LittleFS

در پروژه از سیستم فایل LittleFS برای ذخیره سازی اطلاعات روی حافظه داخلی ESP32 استفاده شده است. استفاده از LittleFS امکان ذخیره اطلاعاتی را فراهم می کند که باید پس از خاموش و روشن شدن دستگاه نیز باقی بمانند. در پروژه حاضر از این قابلیت برای نگهداری اطلاعات مربوط به تنظیمات Provisioning و همچنین سیگنال های ذخیره شده RF و IR استفاده شده است.

4-2-4- فرایند ساخت و بارگذاری Firmware



شکل (4-1) روند ساخت و بارگذاری Firmware

پس از آماده سازی محیط توسعه و نصب وابستگی های تعریف شده، Firmware پروژه توسط PlatformIO ساخته شده و سپس بر روی برد ESP32 بارگذاری می شود.

4-3- پیاده سازی سخت افزار

4-3-1 معرفی ساختار سخت افزاری

در طراحی پیاده سازی شده، برای هر یک از اجزای سخت افزاری پایه مشخصی از GPIO های ESP32 اختصاص داده شده است. نگاشت پایه ها در فایل پیکربندی config.h متمرکز شده تا تغییر اتصالات سخت افزاری بدون نیاز به جست و جو در سایر بخش های Firmware امکان پذیر باشد. جدول (1-3) نگاشت پایه های مورد استفاده را نشان می دهد.

4-3-2 پیاده سازی حسگر دما و رطوبت DHT11

برای اندازه گیری دما و رطوبت محیط از حسگر DHT11 و کتابخانه Adafruit Sensors استفاده شده است. برای راه اندازی حسگر، تابع setupDHT() تعریف شده است:

```
void setupDHT()
{
    dht.begin();
}
```

این تابع در فرایند راه اندازی Firmware فراخوانی شده و ارتباط نرم افزار با حسگر DHT11 را آماده می کند.

برای دریافت رطوبت محیط، تابع readHumidity فراخوانی می شود:

```
float readHumidity()
{
    float input = dht.readHumidity();
    if (isnan(input))
    {
        Serial.println("Failed to read humidity from DHT sensor");
    }
    return input;
}
```

به همین شکل برای دریافت دمای محیط، تابع readTemperature فراخوانی می شود:

```
float readTemperature()
{
    float input = dht.readTemperature();
    if (isnan(input))
    {
        Serial.println("Failed to read temperature from DHT sensor");
    }
    return input;
}
```

در این پیاده سازی، مقدار حسگر بررسی می شود و سپس نتیجه برگردانده می شود.

4-3-3- پیاده سازی حسگر نور LDR

برای اندازه گیری میزان نور محیط از ماژول LDR استفاده شده است. مقدار خروجی با استفاده از تابع `analogRead` دریافت می شود:

```
int readLDR()
{
    int input = analogRead(LDR_PIN);
    return input;
}
```

در نتیجه، ESP32 در زمان نمونه برداری مقدار ADC مربوط به خروجی LDR را دریافت کرده و این مقدار به عنوان داده مربوط به شدت نور در سیستم مورد استفاده قرار می گیرد.

4-3-4- پیاده سازی حسگر MQ-2

برای تشخیص دود و گاز های قابل اشتعال از حسگر MQ-2 استفاده شده است. مقدار خروجی آنالوگ با استفاده از MQ-2 دریافت می شود:

```
int readMQ2()
{
    return analogRead(MQ2_PIN);
}

bool checkThreshold(int value)
{
    if (value > MQ2_THRESHOLD)
    {
        return true;
    }
    return false;
}
```

در این مرحله مقدار خام ADC دریافت می شود و در مرحله بعد عبور از آستانه تشخیص داده می شود. منطق فعال کردن وضعیت هشدار در بخش مدیریت سنسور ها پیاده سازی شده است.

4-3-5- پیاده سازی حسگر تشخیص حرکت PIR

برای تشخیص حرکت از حسگر PIR استفاده شده است.

```
void setupPIR()
{
    pinMode(PIR_PIN, INPUT);

    motionDetectionEnabled = loadMotionDetection();
    if (motionDetectionEnabled)
```

```
{
    Serial.println("Motion detection restored to ON");
}
}
```

در هنگام راه اندازی، پایه مربوط به عنوان ورودی تنظیم می شود و اطلاعات وضعیت فعال بودن تشخیص حرکت بازیابی می شود.

```
int readPIR()
{
    int input = digitalRead(PIR_PIN);
    if (input == HIGH)
    {
        return 1;
    }
    return 0;
}
```

در این پیاده سازی، مقدار HIGH به عنوان تشخیص حرکت و مقدار LOW به عنوان عدم تشخیص حرکت در نظر گرفته می شود.

برای کنترل قابلیت تشخیص حرکت، دو متغیر motionDetectionEnabled و motionDetectionTriggered در Firmware تعریف شده است.

متغیر motionDetectionEnabled وضعیت سیستم تشخیص حرکت را مشخص می کند و متغیر motionDetectionTriggered وضعیت تحریک شدن سیستم را مشخص می کند.

فعال کردن قابلیت تشخیص حرکت توسط تابع turnONMotionDetection انجام می شود. در این تابع علاوه بر تغییر وضعیت داخلی، وضعیت جدید در حافظه ذخیره شده و از طریق MQTT نیز اطلاع رسانی می شود. به صورت مشابه، تابع turnOFFMotionDetection قابلیت تشخیص حرکت را غیر فعال می کند. جزئیات پیاده سازی در فایل src/PIR/pir.cpp پیوست شده، قابل مشاهده است.

4-3-6- پیاده سازی سیستم هشدار صوتی

برای ایجاد هشدار صوتی از یک Active Buzzer استفاده شده است. این بازر با منطق Active-Low کنترل می شود؛ به این معنی که سطح منطقی Low باعث فعال شدن بازر و سطح HIGH باعث غیرفعال شدن آن می شود.

در هنگام راه اندازی، پایه مشخص شده به عنوان خروجی تنظیم شده و وضعیت بازر در حالت خاموش قرار می گیرد:

```
void setupBuzzer()
{
    pinMode(BUZZER_PIN, OUTPUT);
    stopAlarm();
}
```

تابع startAlarm وظیفه فعال کردن هشدار را بر عهده دارد. در مقابل، تابع stopAlarm وظیفه غیرفعال کردن هشدار را بر عهده دارد. در صورت فعال بودن هشدار، وضعیت پایه خروجی بازر به صورت متناوب توسط updateAlarm تغییر می کند.

جزئیات پیاده سازی در فایل src/Alarm/alarm.cpp پیوست شده، قابل مشاهده است.

4-4- پیاده سازی سیستم ارتباط رادیویی RF

4-4-1 معرفی سیستم RF

یکی از قابلیت های اصلی سامانه، امکان دریافت، ذخیره سازی و بازپخش سیگنال های رادیویی RF است. این قابلیت با هدف ایجاد امکان کنترل تجهیزات الکتریکی دارای کنترل از راه دور در سامانه پیاده سازی شده است.

در طراحی فعلی، سیستم RF از دو باند فرکانسی ۳۱۵ و ۴۳۳ مگاهرتز پشتیبانی می کند. از آنجا که گیرنده و فرستنده هر باند به صورت مستقل عمل می کنند، برای هر یک از دو باند یک نمونه مستقل از کلاس RCSwitch در Firmware ایجاد شده است.

4-4-2 راه اندازی سیستم RF

فرایند راه اندازی سیستم RF در تابع setupRF انجام می شود. در این تابع، گیرنده و فرستنده مربوط به هر دو باند فعال می شوند:

```
void setupRF()
{
    rfSwitch433.enableReceive(RF_RX_433_PIN);
    rfSwitch433.enableTransmit(RF_TX_433_PIN);
    rfSwitch315.enableReceive(RF_RX_315_PIN);
    rfSwitch315.enableTransmit(RF_TX_315_PIN);
}
```

بنابراین در زمان راه اندازی دستگاه، هر دو گیرنده برای دریافت سیگنال و هر دو فرستنده برای ارسال سیگنال آماده می شوند.

4-4-3 ساختار داده سیگنال RF

اطلاعات هر سیگنال RF در ساختاری با نام RFSignal ذخیره می شود:

```
struct RFSignal
{
    String name;
    uint32_t code;
    uint16_t bits;
    uint16_t protocol;
    uint16_t pulse;
```

```
uint16_t band;
};
```

این ساختار شامل اطلاعات اصلی مورد نیاز برای شناسایی و بازپخش یک سیگنال RF است که در جدول زیر جزئیات هر متغیر آورده شده است:

جدول (4-1) ساختار داده سیگنال RF

متغیر	نوع داده	کاربرد
name	string	نام سیگنال
code	uint32_t	کد دریافتی سیگنال
bits	uint16_t	تعداد بیت سیگنال
protocol	uint16_t	پروتکل تشخیص داده شده
pulse	uint16_t	طول پالس
band	uint16_t	باند فرکانسی

همچنین آرایه signals برای ذخیره سیگنال های دریافت شده تعریف شده است.

4-4-4 دریافت سیگنال RF

فرایند دریافت سیگنال توسط تابع readRFSignal انجام می شود. این تابع، وضعیت هر دو گیرنده را بررسی می کند.

در ابتدا یک اشاره گر به گیرنده فعال و متغیری برای نگهداری باند فرکانسی تعریف می شود. سپس گیرنده ۴۳۳ مگاهرتز بررسی می شود. در صورتی که گیرنده ۴۳۳ مگاهرتز سیگنالی دریافت نکرده باشد، گیرنده ۳۱۵ مگاهرتز بررسی می شود.

پس از شناسایی گیرنده فعال، پارامتر های سیگنال از طریق API کتابخانه RCSSwitch استخراج می شوند. در صورت معتبر بودن سیگنال، اطلاعات آن در اولین موقعیت آزاد آرایه signals ذخیره می شود. پس از پردازش نیز وضعیت گیرنده به حالت اولیه بازگردانده می شود تا آماده دریافت سیگنال بعدی باشد.

جزئیات پیاده سازی در فایل های src/RF/receiver.cpp و src/RF/general.cpp پیوست شده، قابل مشاهده است.

4-4-5 مدیریت فرایند دریافت سیگنال

مدیریت فرایند دریافت و ذخیره سیگنال در تابع handleRFReceive انجام می شود. ابتدا قرار داشتن سیستم در حالت دریافت بررسی می شود. در صورت دریافت موفق سیگنال، وضعیت دریافت به حالت غیرفعال تغییر می کند و اطلاعات دریافت شده در حافظه دائمی سیستم ذخیره می شود. جزئیات پیاده سازی در فایل src/RF/receiver.cpp پیوست شده، قابل مشاهده است.

4-4-6- بازپخش سیگنال RF

برای ارسال مجدد سیگنال ذخیره شده از تابع `transmitRFSignal` استفاده شده است. پیش از ارسال، وجود سیگنال و اعتبار سیگنال و پارامترهای اصلی آن بررسی می شود. سپس با توجه به باند فرکانسی آن، فرستنده مناسب انتخاب می شود. در ابتدا، پروتکل و طول پالس ذخیره شده برای فرستنده مربوطه منتقل می شود. سپس کد و تعداد بیت سیگنال ارسال می شود. جزئیات پیاده سازی در فایل `src/RF/transmitter.cpp` پیوست شده، قابل مشاهده است.

4-4-7- مدیریت سیگنال های ذخیره شده

برای نمایش سیگنال های ذخیره شده، تابع `listRFSignals` پیاده سازی شده است. در صورت وجود داشتن سیگنال، اطلاعات هر سیگنال شامل نام، باند، کد، تعداد بیت و پروتکل از طریق `Serial` نمایش داده می شود و همچنین به صورت یک شی `JSON` ساخته شده و از طریق `MQTT` منتشر می شود. `Topic` مربوط به این اطلاعات `RF/list` تعریف شده است که امکان مدیریت سیگنال ها را از طریق داشبورد نیز فراهم می کند. جزئیات پیاده سازی در فایل `src/RF/general.cpp` پیوست شده، قابل مشاهده است.

4-4-8- جست و جو سیگنال RF

```

RFSignal *findRFSignalByName(const String &name)
{
    for (size_t i = 0; i < signalCount; i++)
    {
        if (signals[i].name == name)
        {
            return &signals[i];
        }
    }
    return nullptr;
}

```

برای پیدا کردن یک سیگنال RF بر اساس اسم آن، تابع `findRFSignalByName` پیاده سازی شده است. این تابع، در تمام سیگنال های ذخیره شده جست و جو می کند و در صورت پیدا کردن سیگنال با اسم داده شده، آن را بر می گرداند.

4-4-9- حذف سیگنال ذخیره شده

```

RFSignal *signal = findRFSignalByName(name);
if (signal == nullptr)
{
    Serial.println("RF signal not found");
    return;
}

size_t index = signal - signals;
for (size_t i = index; i + 1 < signalCount; i++)
{
    signals[i] = signals[i + 1];
}
signalCount--;

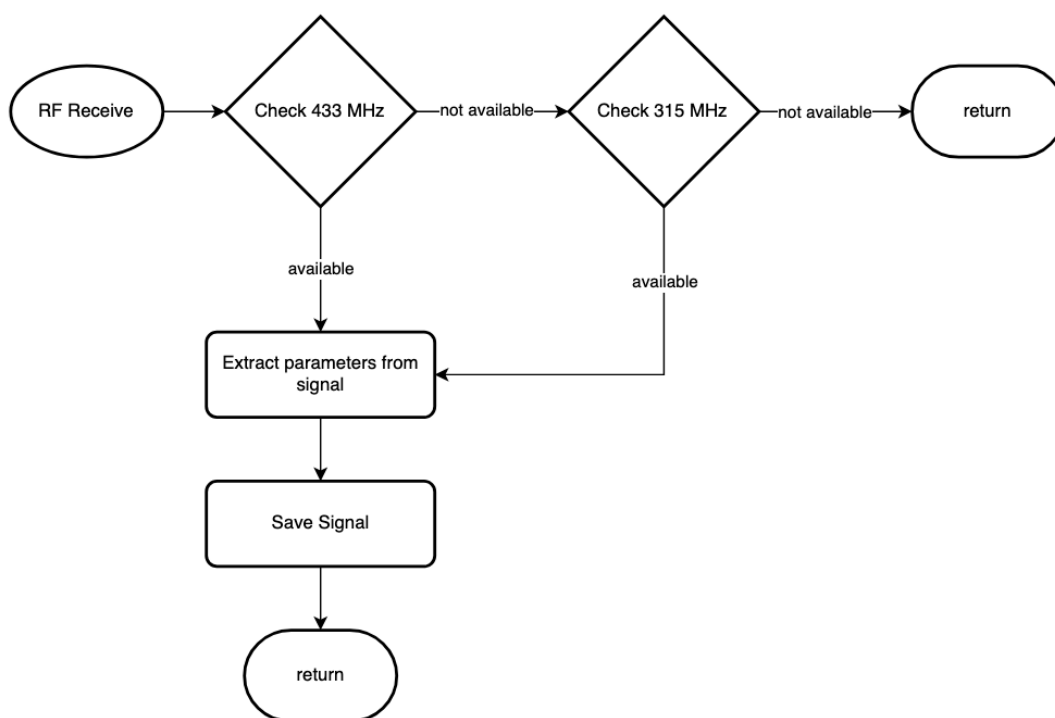
if (saveRFSignals())
{
    Serial.print("Deleted RF signal: ");
    Serial.println(name);
}
else
{
    loadRFSignals();
    Serial.println("Failed to delete RF signal");
}

```

برای حذف سیگنال ذخیره شده، در ابتدا وجود سیگنال مورد نظر بررسی می شود. سپس index آن پیدا شده و index های بعد آن در آرایه signals، به index قبلی خود شیفت داده می شود و سپس سیگنال ها ذخیره می شود.

4-4-10- منطق کلی دریافت و ارسال RF

فرایند دریافت سیگنال در سامانه به صورت زیر است:



شکل (4-2) فرایند دریافت سیگنال RF

فرایند ارسال سیگنال در سامانه به صورت زیر است:



شکل (4-3) فرایند ارسال سیگنال RF

4-5- پیاده سازی سیستم مادون قرمز IR

4-5-1- معرفی سیستم مادون قرمز

یکی دیگر از قابلیت های سامانه، امکان دریافت، ذخیره سازی و بازپخش فرمان های مادون قرمز است. این قابلیت با هدف فراهم کردن امکان کنترل تجهیزاتی که از کنترل مادون قرمز استفاده می کنند. در این بخش از یک گیرنده مادون قرمز برای دریافت سیگنال و چهار IR LED برای ارسال مجدد آن استفاده شده است. پردازش سیگنال توسط کتابخانه IRremoteESP8266 انجام می شود.

4-5-2- راه اندازی سیستم IR

برای مدیریت سیستم مادون قرمز، دو شی از کلاس های IRrecv و IRsend ایجاد شده است. گیرنده وظیفه دریافت و decode کردن سیگنال های مادون قرمز را و فرستنده وظیفه encode کردن و ارسال سیگنال مادون قرمز را بر عهده دارد. راه اندازی این دو بخش در تابع setupIR انجام می شود:

```
void setupIR()
{
    irReceiver.enableIRIn();
    irTransmitter.begin();
}
```

4-5-3- ساختار داده سیگنال IR

برای ذخیره اطلاعات سیگنال های مادون قرمز، ساختار IRSignal تعریف شده است:

```
struct IRSignal
{
    String name;
    decode_type_t protocol;
    uint64_t value;
    uint16_t bits;
    uint16_t rawLength;
    uint16_t rawData[MAX_IR_RAW_LENGTH];
};
```

این ساختار شامل اطلاعات اصلی مورد نیاز برای شناسایی و بازپخش یک سیگنال IR است که در جدول زیر جزئیات هر متغیر آورده شده است:

جدول (4-2) ساختار داده سیگنال IR

متغیر	نوع داده	کاربرد
name	string	نام سیگنال
protocol	decode_type_t	پروتکل تشخیص داده شده
value	uint64_t	مقدار Decode شده سیگنال
bits	uint16_t	تعداد بیت سیگنال
raw length	uint16_t	تعداد نمونه داده خام
raw data	uint16_t[]	داده خام سیگنال

برای ذخیره سیگنال های مادون قرمز نیز آرایه IRSignals تعریف شده است.

4-5-4- دریافت و Decode سیگنال IR

دریافت سیگنال مادون قرمز توسط تابع `readIRSignal` انجام می شود. در ابتدا یک ساختار `decode_results` برای دریافت نتیجه `Decode` ایجاد شده و گیرنده تلاش می کند سیگنال موجود را دریافت کند. پس از دریافت سیگنال، دو وضعیت `overflow` و `rawlength` برای معتبر بودن داده بررسی می شود. پس از معتبر بودن سیگنال، یک `index` از آرایه `IRSignals` به سیگنال اختصاص داده می شود و سپس پارامترهای دریافت شده از سیگنال در آن ذخیره می شود. علاوه بر اطلاعات `Decode` شده، داده خام سیگنال نیز ذخیره می شود؛ زیرا همه سیگنال های مادون قرمز الزاما توسط کتابخانه به یک پروتکل شناخته شده قابل `Decode` کردن نیستند و برای بازپخش این سیگنال ها به داده خام آن نیاز است. جزئیات پیاده سازی در فایل های `src/IR/receiver.cpp` و `src/IR/general.cpp` پیوست شده، قابل مشاهده است.

4-5-5- مدیریت فرایند دریافت سیگنال

تابع `handleIRReceive`، فرایند دریافت و ذخیره سیگنال مادون قرمز را مدیریت می کند. ابتدا قرار داشتن سیستم در حالت دریافت بررسی می شود. در صورت دریافت موفق سیگنال، وضعیت دریافت به حالت غیرفعال تغییر می کند و اطلاعات دریافت شده در حافظه دائمی سیستم ذخیره می شود. جزئیات پیاده سازی در فایل `src/IR/receiver.cpp` پیوست شده، قابل مشاهده است.

4-5-6- بازپخش سیگنال IR

فرایند ارسال سیگنال توسط تابع `transmitIRSignal` انجام می شود. در ابتدا، وجود و اعتبار سیگنال بررسی می شود. سپس با توجه به پروتکل ذخیره شده، روش ارسال مناسب انتخاب می شود و سیگنال ارسال می شود. در حال حاضر، `Firmware` از پروتکل های `SONY`، `NEC`، `SAMSUNG` و `PANASONIC` پشتیبانی می کند. در صورت پشتیبانی نشدن و یا ناشناس بودن پروتکل سیگنال، داده خام سیگنال با فرکانس ۳۸ کیلوهرتز بازپخش می شود. جزئیات پیاده سازی در فایل `src/IR/transmitter.cpp` پیوست شده، قابل مشاهده است.

4-5-7- مدیریت سیگنال های ذخیره شده

برای نمایش سیگنال های ذخیره شده، تابع `listIRSignals` پیاده سازی شده است.

در صورت وجود داشتن سیگنال، اطلاعات هر سیگنال شامل نام، پروتکل، تعداد بیت و طول داده خام از طریق Serial نمایش داده می شود و همچنین به صورت یک شی JSON ساخته شده و از طریق MQTT منتشر می شود.

Topic مربوط به این اطلاعات IR/list تعریف شده است که امکان مدیریت سیگنال ها را از طریق داشبورد نیز فراهم می کند.

جزئیات پیاده سازی در فایل src/IR/general.cpp پیوست شده، قابل مشاهده است.

4-5-8- جست و جو سیگنال IR

```
IRSignal *findIRSignalByName(const String &name)
{
    for (size_t i = 0; i < irSignalCount; i++)
    {
        if (irSignals[i].name == name)
        {
            return &irSignals[i];
        }
    }
    return nullptr;
}
```

برای پیدا کردن یک سیگنال IR بر اساس اسم آن، تابع findIRSignalByName پیاده سازی شده است. این تابع، در تمام سیگنال های ذخیره شده جست و جو می کند و در صورت پیدا کردن سیگنال با اسم داده شده، آن را بر می گرداند.

4-5-9- حذف سیگنال ذخیره شده

```
IRSignal *signal = findIRSignalByName(name);
if (signal == nullptr)
{
    Serial.println("IR signal not found");
    return;
}

size_t index = signal - irSignals;
for (size_t i = index; i + 1 < irSignalCount; i++)
{
    irSignals[i] = irSignals[i + 1];
}
irSignalCount--;

if (saveIRSignals())
{
    Serial.print("Deleted IR signal: ");
```

```

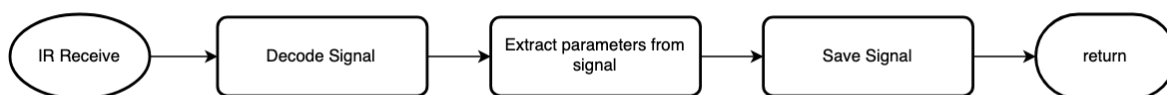
        Serial.println(name);
    }
    else
    {
        loadIRSignals();
        Serial.println("Failed to delete IR signal");
    }
}

```

برای حذف سیگنال ذخیره شده، در ابتدا وجود سیگنال مورد نظر بررسی می شود. سپس index آن پیدا شده و index های بعد آن در آرایه irSignals، به index قبلی خود شیفت داده می شود و سپس سیگنال ها ذخیره می شود.

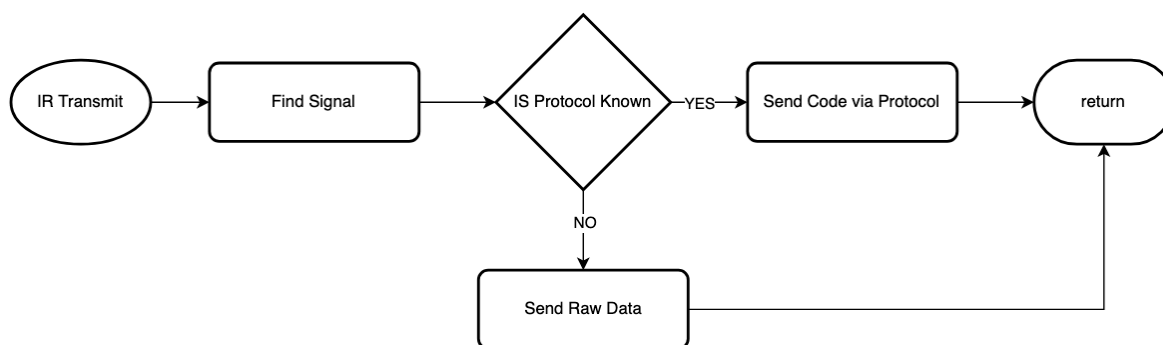
10-5-4- منطق کلی دریافت و ارسال IR

فرایند دریافت سیگنال در سامانه به صورت زیر است:



شکل (4-4) فرایند دریافت سیگنال IR

فرایند ارسال سیگنال در سامانه به صورت زیر است:



شکل (4-5) فرایند ارسال سیگنال IR

4-6- پیاده سازی ارتباط Wi-Fi و MQTT

1-4-6- مقدمه

به منظور ایجاد ارتباط میان دستگاه و زیرساخت نرم افزاری سامانه، از دو فناوری Wi-Fi و MQTT استفاده

شده است. ارتباط Wi-Fi وظیفه اتصال ESP32 به شبکه محلی و فراهم کردن دسترسی به MQTT Broker را بر عهده دارد و پروتکل MQTT نیز برای تبادل پیام های کنترلی و اطلاعات حسگرها استفاده می شود.

4-6-2- ایجاد MQTT Client

```
static WiFiClient wifiClient;
static PubSubClient mqttClient(wifiClient);
static MqttCommandHandler mqttCommandHandler = nullptr;
```

در ابتدای مازول MQTT، یک شی از کلاس WiFiClient و یک شی از کلاس PubSubClient ایجاد شده است. همچنین برای مدیریت فرمان های دریافتی از MQTT، یک تابع Callback از نوع MqttCommandHandler تعریف شده است.

4-6-3- برقراری ارتباط Wi-Fi

```
static void connectWiFi()
{
    if (WiFi.status() == WL_CONNECTED)
    {
        return;
    }

    const char *ssid = provisioningConfig.wifiSSID.length() > 0
        ? provisioningConfig.wifiSSID.c_str()
        : WIFI_SSID;
    const char *wpass = provisioningConfig.wifiPassword.length() > 0
        ? provisioningConfig.wifiPassword.c_str()
        : WIFI_PASSWORD;

    WiFi.begin(ssid, wpass);
}
```

وظیفه اتصال دستگاه به شبکه Wi-Fi توسط تابع connectWiFi انجام می شود. ابتدا وضعیت فعلی اتصال بررسی می شود و در صورتی که دستگاه از قبل متصل باشد، عملیات اتصال مجدد انجام نمی شود. سپس اطلاعات شبکه از تنظیمات Provisioning یا مقادیر پیش فرض خوانده می شود و در نهایت اتصال به شبکه Wi-Fi انجام می شود.

4-6-4- اتصال به MQTT Broker

```
static bool connectMQTT()
{
```

```

const char *host = provisioningConfig.mqttHost.length() > 0
    ? provisioningConfig.mqttHost.c_str()
    : MQTT_HOST;
uint16_t port = provisioningConfig.mqttPort > 0
    ? provisioningConfig.mqttPort
    : MQTT_PORT;
const char *mqttUser = provisioningConfig.mqttUser.length() > 0
    ? provisioningConfig.mqttUser.c_str()
    : MQTT_USER;
const char *mqttPass = provisioningConfig.mqttPassword.length() > 0
    ? provisioningConfig.mqttPassword.c_str()
    : MQTT_PASSWORD;

bool connected = false;
if (strlen(mqttUser) > 0)
{
    connected = mqttClient.connect(MQTT_CLIENT_ID, mqttUser, mqttPass);
}
else
{
    connected = mqttClient.connect(MQTT_CLIENT_ID);
}

if (!connected)
{
    Serial.print("MQTT connect failed, state=");
    Serial.println(mqttClient.state());
    return false;
}

String commandTopic = topicFor(MQTT_COMMAND_TOPIC);
mqttClient.subscribe(commandTopic.c_str());
return true;

```

پس از برقراری ارتباط Wi-Fi، فرایند اتصال به MQTT Broker توسط تابع connectMQTT انجام می شود. در صورت برقرار نبودن ارتباط Wi-Fi، اتصال MQTT انجام نمی شود. ابتدا اطلاعات Broker از تنظیمات Provisioning و در صورت نبود آن از مقادیر پیش فرض خوانده می شود. ساختار پیاده سازی امکان استفاده از نام کاربری و رمز عبور را نیز برای اتصال فراهم کرده است. سپس با توجه به وجود یا عدم وجود نام کاربری، یکی از دو روش اتصال استفاده می شود. پس از اتصال موفق به Broker، دستگاه به Topic مربوط به دریافت فرمان Subscribe می کند.

4-6-5- راه اندازی اولیه MQTT

```

void setupMQTT(MqttCommandHandler commandHandler)
{
    mqttCommandHandler = commandHandler;
}

```

```

const char *host = provisioningConfig.mqttHost.length() > 0
    ? provisioningConfig.mqttHost.c_str()
    : MQTT_HOST;
uint16_t port = provisioningConfig.mqttPort > 0
    ? provisioningConfig.mqttPort
    : MQTT_PORT;

WiFi.setAutoReconnect(true);
connectWiFi();
mqttClient.setServer(host, port);
mqttClient.setCallback(mqttCallback);
}

```

برای راه اندازی اولیه MQTT از تابع setupMQTT استفاده شده است. ابتدا commandHandler که به عنوان ورودی داده می شود، ذخیره می شود. سپس اطلاعات Broker از تنظیمات Provisioning یا مقادیر پیش فرض خوانده می شود. سپس اتصال به شبکه Wi-Fi انجام می شود و در آخر به MQTT Broker متصل می شود.

4-6-6- برقراری اتصال مجدد MQTT

```

void handleMQTT()
{
    if (WiFi.status() != WL_CONNECTED)
    {
        unsigned long now = millis();
        if (now - lastReconnectAttempt >= MQTT_RECONNECT_INTERVAL)
        {
            lastReconnectAttempt = now;
            connectMQTT();
        }
        return;
    }

    if (!mqttClient.connected())
    {
        unsigned long now = millis();
        if (now - lastReconnectAttempt >= MQTT_RECONNECT_INTERVAL)
        {
            lastReconnectAttempt = now;
            connectMQTT();
        }
        return;
    }

    mqttClient.loop();
}

```

در طول اجرای برنامه، اتصال دستگاه به شبکه Wi-Fi و Broker چک می شود و در صورت وجود داشتن

ایراد، تلاش برای اتصال مجدد انجام می شود. در غیر اینصورت، دریافت پیام جدید و اجرای Callback های ورودی را برعهده دارد.

4-6-7- دریافت فرمان های MQTT

```
static void mqttCallback(char *topic, byte *payload, unsigned int length)
{
    String command;
    command.reserve(length + 1);

    for (unsigned int i = 0; i < length; i++)
    {
        command += static_cast<char>(payload[i]);
    }

    command.trim();
    Serial.print("MQTT command on ");
    Serial.print(topic);
    Serial.print(": ");
    Serial.println(command);

    if (command.length() > 0 && mqttCommandHandler != nullptr)
    {
        mqttCommandHandler(command);
    }
}
```

برای دریافت پیام های MQTT از Callback استفاده شده است. در این تابع Payload دریافتی به یک String تبدیل می شود و سپس فرمان در Serial Monitor نمایش داده می شود و در نهایت فرمان، به Handler مربوطه منتقل می شود.

4-6-8- انتشار اطلاعات در MQTT

```
bool publishMQTT(const String &subTopic, const String &payload, bool retained)
{
    if (!mqttClient.connected())
    {
        return false;
    }

    String topic = topicFor(subTopic);
    return mqttClient.publish(topic.c_str(), payload.c_str(), retained);
}
```

برای ارسال اطلاعات به Broker، تابع publishMQTT پیاده سازی شده است:

در ابتدا اتصال MQTT بررسی می شود و سپس Topic نهایی ساخته شده و با Payload منتشر می شود.

4-7- پیاده سازی پیکربندی اولیه و مدیریت اتصال به شبکه

به منظور جلوگیری از وابستگی مستقیم تنظیمات شبکه و MQTT Broker به کد Firmware، در پروژه یک زیر سامانه مستقل با عنوان Provisioning طراحی و پیاده سازی شده است. وظیفه این زیرسامانه، مدیریت اطلاعات موردنیاز برای اتصال ESP32 به شبکه Wi-Fi و MQTT Broker و همچنین فراهم کردن امکان تغییر این تنظیمات بدون نیاز به تغییر یا کامپایل مجدد Firmware است.

4-7-1- ساختار اطلاعات پیکربندی

در ماژول Provisioning، اطلاعات ارتباطی دستگاه با نام ProvisioningConfig در قالب ساختار زیر تعریف شده است:

```
struct ProvisioningConfig
{
    String wifiSSID;
    String wifiPassword;
    String mqttHost;
    uint16_t mqttPort;
    String mqttUser;
    String mqttPassword;
    String client;
};
```

این ساختار امکان نگهداری تمامی پارامترهای موردنیاز برای برقراری ارتباط دستگاه با شبکه و MQTT Broker را فراهم می کند. جزئیات هر متغیر در جدول زیر آمده است:

جدول (4-3) ساختار داده تنظیمات پیکربندی

متغیر	نوع داده	کاربرد	
wifiSSID	String	نام AP	اجباری
wifiPassword	String	رمز عبور Wi-Fi	اجباری
mqttHost	String	آدرس MQTT Broker	اجباری
mqttPort	uint16_t	پورت MQTT Broker	اجباری
mqttUser	String	نام کاربری MQTT Broker	اختیاری
mqttPassword	String	رمز عبور MQTT Broker	اختیاری
client	String	نام دستگاه	اجباری

4-7-2 AP اختصاصی برای پیکربندی

یکی از قابلیت های پیاده سازی شده در پروژه، ایجاد یک AP اختصاصی برای انجام تنظیمات دستگاه است. برد ESP32 در حالت WIFI_AP_STA فعالیت می کند. بنابراین دستگاه می تواند به صورت همزمان به شبکه اصلی متصل باشد و یک شبکه بی سیم اختصاصی نیز برای پیکربندی ایجاد کند.

4-7-3 پورتال تحت وب پیکربندی

برای انجام تنظیمات، یک پورتال تحت وب روی ESP32 اجرا می شود. کاربر می تواند با اتصال رایانه یا تلفن همراه خود به شبکه Intelligence-Setup و مراجعه به آدرس 192.168.4.1 به صفحه پیکربندی دسترسی پیدا کند.

4-7-4 پیاده سازی Captive Portal

برای ساده تر شدن فرایند پیکربندی، در پروژه از مکانیزم Captive Portal استفاده شده است. برای این منظور یک DNS Server ایجاد شده و تمامی درخواست های DNS به آدرس IP مربوط به Access Point هدایت می شوند. همچنین یک WebServer روی پورت ۸۰ ایجاد شده است.

4-7-5 مدیریت درخواست های HTTP

```
server.on("/", HTTP_GET, handleRoot);
// OS connectivity-check probes – redirecting them (instead of returning the
// expected "success"/204 response) is what makes the captive portal pop up
// automatically on iOS / Android / Windows.
server.on("/generate_204", HTTP_GET, redirectToPortal);           // Android
server.on("/gen_204", HTTP_GET, redirectToPortal);               // Android (alt)
server.on("/hotspot-detect.html", HTTP_GET, redirectToPortal);   // Apple
server.on("/ncsi.txt", HTTP_GET, redirectToPortal);               // Windows
server.on("/connecttest.txt", HTTP_GET, redirectToPortal);       // Windows
server.on("/redirect", HTTP_GET, redirectToPortal);               // Windows
server.on("/save", HTTP_POST, handleSave);
server.onNotFound(handleNotFound);
```

در ماژول Provisioning مسیر های مختلفی برای مدیریت درخواست ای HTTP تعریف شده است. مهم ترین مسیر، مسیر اصلی / است که صفحه پیکربندی را به کاربر نمایش می دهد. همچنین مسیر /save برای دریافت اطلاعات ارسال شده توسط کاربر مورد استفاده قرار گرفته است.

در صورت دریافت درخواست برای مسیر های ناشناخته نیز کاربر به صفحه پیکربندی هدایت می شود. برای بهتر شدن تجربه کاربری، Connectivity Check های سیستم های عامل به صفحه پیکربندی هدایت

می شوند تا به صورت خودکار بعد از متصل شدن کاربر به AP، صفحه پیکربندی باز شود.

4-7-6- دریافت و اعتبارسنجی تنظیمات

تابع `handleSave` اطلاعات وارد شده توسط کاربر را از درخواست HTTP دریافت کرده و یک نمونه جدید از `ProvisioningConfig` ایجاد می کند. پارامترهای مربوط به Wi-Fi و MQTT از فرم دریافت شده و سپس بررسی می شوند. در صورتی که نام شبکه، آدرس MQTT Broker یا نام دستگاه خالی باشد، درخواست با خطای 400 رد می شود. در صورتی که کاربر در صفحه تنظیمات، فیلد رمز عبور Wi-Fi یا MQTT را خالی بگذارد، برنامه مقدار قبلی را حفظ می کند. در نتیجه، رمز عبور موجود در حافظه به صورت مستقیم به کاربر نمایش داده نمی شود و کاربر برای حفظ رمز عبور فعلی نیازی به وارد کردن مجدد آن ندارد. پس از معتبر بودن اطلاعات، تنظیمات توسط `saveProvisioningConfig` ذخیره می شود و دستگاه مجدداً راه اندازی می شود. جزئیات پیاده سازی در فایل `src/Provisioning/provisioning.cpp` پیوست شده، قابل مشاهده است.

4-8- پیاده سازی مدیریت حافظه و ذخیره سازی اطلاعات

در این پروژه، به منظور حفظ اطلاعات موردنیاز سیستم پس از خاموش و روشن شدن ESP32، از سیستم فایل LittleFS استفاده شده است. این مازول مسئولیت ذخیره سازی و بازیابی اطلاعات مربوط به تنظیمات سیستم، وضعیت تشخیص حرکت و سیگنال های IR و RF را بر عهده دارد.

4-8-1- سیستم فایل LittleFS

برای ذخیره سازی اطلاعات از کتابخانه LittleFS استفاده شده است. در ابتدای عملیات دسترسی به حافظه، تابع داخلی `ensureLittleFS` وضعیت آماده بودن سیستم فایل را بررسی می کند.

4-8-2- قالب JSON

برای ساختاردهی اطلاعات ذخیره شده از کتابخانه `ArduinoJson` استفاده شده است. اطلاعات پیش از ذخیره سازی به ساختار JSON تبدیل شده و هنگام بازیابی مجدداً از JSON به ساختارهای مورد استفاده برنامه تبدیل می شوند.

4-8-3- ذخیره و بازیابی وضعیت تشخیص حرکت

از آنجا که وضعیت فعال یا غیرفعال بودن تشخیص حرکت باید پس از راه اندازی مجدد دستگاه حفظ شود، این مقدار در فایل settings.json/ ذخیره می شود. تابع saveMotionDetection مقدار وضعیت تشخیص حرکت را دریافت کرده و آن را در قالب JSON ذخیره می کند. در مقلبل، تابع loadMotionDetection فایل تنظیمات را باز کرده و مقدار ذخیره شده را بازیابی می کند. در صورتی که فایل وجود نداشته باشد، سیستم به صورت پیش فرض قابلیت تشخیص حرکت را غیرفعال می کند. این اطلاعات در زمان اجرای setupPIR مورد استفاده قرار گرفته و وضعیت قبلی تشخیص حرکت را بازیابی می کند. جزئیات پیاده سازی در فایل src/Mememory/mememory.cpp پیوست شده، قابل مشاهده است.

4-8-4- ذخیره و بازیابی سیگنال های RF

سیگنال های RF دریافت شده توسط سیستم در فایل rf_signals.json/ ذخیره می شوند. تابع saveRFSignals ابتدا یک آرایه JSON ایجاد کرده و سپس برای هر سیگنال، یک شی JSON ایجاد کرده و اطلاعات سیگنال های موجود را در آن قرار می دهد. در نهایت، آرایه JSON را ذخیره می کند. تابع loadRFSignals هنگام راه اندازی سیستم، اطلاعات ذخیره شده در سیستم را بازیابی می کند و در آرایه signals که در بخش 3-4-4- توضیح داده شده است، قرار می دهد. جزئیات پیاده سازی در فایل src/Mememory/mememory.cpp پیوست شده، قابل مشاهده است.

4-8-5- ذخیره و بازیابی سیگنال های IR

سیگنال های IR دریافت شده توسط سیستم در فایل ir_signals.json/ ذخیره می شوند. تابع saveIRSignals ابتدا یک آرایه JSON ایجاد کرده و سپس برای هر سیگنال، یک شی JSON ایجاد کرده و اطلاعات سیگنال های موجود را در آن قرار می دهد. در نهایت، آرایه JSON را ذخیره می کند. تابع loadIRSignals هنگام راه اندازی سیستم، اطلاعات ذخیره شده در سیستم را بازیابی می کند و در آرایه irSignals که در بخش 3-4-5- توضیح داده شده است، قرار می دهد. جزئیات پیاده سازی در فایل src/Mememory/mememory.cpp پیوست شده، قابل مشاهده است.

4-8-6- ذخیره و بازیابی تنظیمات Provisioning

اطلاعات مربوط به پیکربندی اولیه دستگاه در فایل provisioning.json/ ذخیره می شوند.

تابع `saveProvisioningConfig`، یک شی JSON ساخته و تنظیمات را در آن قرار می دهد و سپس آن را ذخیره می کند. تابع `loadProvisioningConfig`، تنظیمات را از فایل خوانده و در `provisioningConfig` که در بخش 1-7-4- توضیح داده شده است، قرار می دهد. جزئیات پیاده سازی در فایل `src/Mememory/mememory.cpp` پیوست شده، قابل مشاهده است.

4-9- راه اندازی و هماهنگی ماژول ها

هسته اصلی برنامه شامل دو تابع `setup` و `loop` است. تابع `setup` تنها یک بار در هنگام راه اندازی ESP32 اجرا می شود و وظیفه آماده سازی اولیه سیستم را برعهده دارد. پس از آن، تابع `loop` به صورت مداوم اجرا شده و وظایف مختلف سیستم را مدیریت می کند.

4-9-1- تابع `setup`

```
void setup()
{
    Serial.begin(115200);
    delay(200);

    loadProvisioningConfig();
    startConfigAP();

    setupSensors();
    setupMQTT(handleMqttCommand);
    printHelp();
}
```

تابع `setup` نقطه شروع راه اندازی سیستم است و تنها یک بار پس از روشن شدن دستگاه اجرا می شود. در ابتدا ارتباط سریال با نرخ 115200 baud راه اندازی می شود. سپس تنظیمات ذخیره شده `Provisioning` از حافظه داخلی بازیابی شده و `Access Point` مربوط به سامانه پیکربندی فعال می شود. پس از آن، تابع `setupSensors` وظیفه راه اندازی اجزای سخت افزاری سیستم را بر عهده می گیرد. در ادامه ارتباط `MQTT` تنظیم شده و در نهایت فرمان های قابل استفاده در `Serial Monitor` نمایش داده می شوند.

4-9-2- تابع `loop`

```
void loop()
{
    handleSerial();
    handleMQTT();
    handleConfigAP();
}
```

```
handleSensors();
}
```

پس از تکمیل عملیات راه اندازی، تابع loop به صورت مداوم اجرا می شود. در هر چرخه اصلی برنامه، چهار وظیفه اصلی پردازش می شوند:

- دریافت و پردازش فرمان های Serial
- مدیریت ارتباط MQTT
- پردازش درخواست های Captive Portal
- مدیریت سنسور ها، هشدار ها و سیگنال های RF و IR

3-9-4- راه اندازی ماژول های سخت افزاری

```
void setupSensors()
{
    setupMQ2();
    setupDHT();
    setupPIR();
    setupBuzzer();
    setupRF();
    setupIR();
    setupLDR();
    nextMQ2ReadAt = millis() + MQ2_PREHEAT_DURATION;
    loadRFSignals();
    loadIRSignals();
    setupStatusLED();
}
```

تابع setupSensors در ماژول Sensors وظیفه آماده سازی تمام سنسور ها و تجهیزات جانبی را بر عهده دارد.

3-9-4- مدیریت فرمان های سیستم

فرمان های سیستم در ماژول Serial پردازش می شوند. این فرمان ها از طریق MQTT و Serial Monitor قابل دریافت هستند.

1-4-9-4- پردازش فرمان های RF

فرمان های مربوط به RF شامل چهار عملیات اصلی هستند:

rf rx <name>

rf tx <name>

rf del <name>

rf list

در صورت دریافت فرمان *rf rx <name>*، ابتدا وجود نام سیگنال بررسی می شود. در صورت معتبر بودن

درخواست، سیستم وارد وضعیت دریافت RF می شود.
 در صورت دریافت فرمان `rf tx <name>` ابتدا سیگنال با نام مورد نظر پیدا می شود و سپس سیگنال برای مازول RF ارسال می شود.
 جزئیات پیاده سازی در فایل `src/Serial/serial.cpp` پیوست شده، قابل مشاهده است.

4-9-4-2- پردازش فرمان های IR

فرمان های مربوط به IR شامل چهار عملیات اصلی هستند:

`ir rx <name>`

`ir tx <name>`

`ir del <name>`

`ir list`

در صورت دریافت فرمان `ir rx <name>` ابتدا وجود نام سیگنال بررسی می شود. در صورت معتبر بودن درخواست، سیستم وارد وضعیت دریافت IR می شود.
 در صورت دریافت فرمان `ir tx <name>` ابتدا سیگنال با نام مورد نظر پیدا می شود و سپس سیگنال برای مازول IR ارسال می شود.
 جزئیات پیاده سازی در فایل `src/Serial/serial.cpp` پیوست شده، قابل مشاهده است.

4-9-4-3- مدیریت فرمان های سیستم هشدار

```
if (action == "buzzer")
{
    int subSeparator = argument.indexOf(' ');
    String subAction = subSeparator == -1 ? argument : argument.substring(0,
subSeparator);
    String name = subSeparator == -1 ? "" : argument.substring(subSeparator +
1);

    if (subAction == "on")
    {
        startAlarm();
        return;
    }

    if (subAction == "off")
    {
        stopAlarm();
        return;
    }

    Serial.println("Unknown command. Type: help");
    return;
}
```

سیستم هشدار فرمان های زیر را برای کنترل مستقیم Buzzer در اختیار کاربر قرار می دهد:

`buzzer on`

`buzzer off`

علاوه بر کنترل دستی، سیستم هشدار می تواند توسط رویداد های حسگر ها نیز فعال شود. برای قطع کردن هشدار پس از وقوع یک رویداد، از فرمان buzzer off می توان استفاده کرد.

4-9-4-4- مدیریت فرمان های تشخیص حرکت

```
if (action == "motion")
{
    int subSeparator = argument.indexOf(' ');
    String subAction = subSeparator == -1 ? argument : argument.substring(0,
subSeparator);
    String name = subSeparator == -1 ? "" : argument.substring(subSeparator +
1);

    if (subAction == "on")
    {
        turnONMotionDetection();
        return;
    }

    if (subAction == "off")
    {
        turnOFFMotionDetection();
        return;
    }

    Serial.println("Unknown command. Type: help");
    return;
}
```

تشخیص حرکت نیز از طریق فرمان های زیر قابل فعال یا غیرفعال کردن است:

motion on

motion off

در صورت فعال یا غیرفعال شدن قابلیت تشخیص حرکت، وضعیت آن در حافظه داخلی دستگاه نیز ذخیره می شود.

4-9-5- مدیریت خواندن حسگرها

```
void handleSensors()
{
    handleRFReceive(receivingRF, receiveName);
    handleIRReceive(receivingIR, receiveName);
    updateAlarm();
    updateStatusLED();
    updatePulseLED();
    checkMotionDetection();
    printSensors();
}
```

خواندن حسگرها در ماژول Sensors توسط تابع handleSensors انجام می شود.

در این تابع ابتدا وضعیت دریافت RF و IR بررسی می شود. سپس سیستم هشدار و LED به روزرسانی شده و وضعیت تشخیص حرکت بررسی می شود. در نهایت اطلاعات حسگرها خوانده و منتشر می شوند.

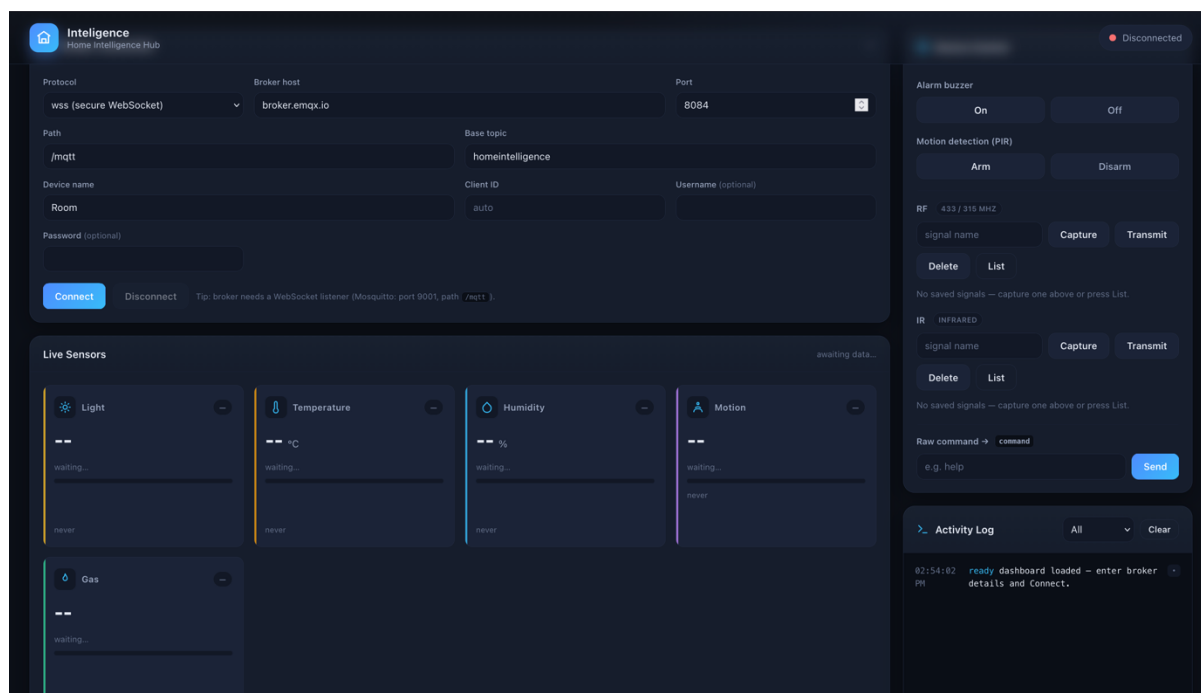
1-5-9-4- زمان بندی خواندن حسگرها

برای جلوگیری از خواندن بیش از حد حسگرها و ارسال مداوم اطلاعات به Broker، فاصله زمانی ۱۰ ثانیه بین هر خوانش در نظر گرفته شده است.

به دلیل نیاز به پیش گرمایش و همچنین متفاوت بودن فاصله زمانی برای خواندن حسگر MQ-2، از تابع printMQ2 استفاده شده است.

در صورت رسیدن زمان، مقادیر LDR، دما، رطوبت و PIR خوانده شده و از طریق MQTT منتشر می شود. جزئیات پیاده سازی در فایل src/Sensors/sensors.cpp پیوست شده، قابل مشاهده است.

10-4- پیاده سازی داشبورد وب و رابط کاربری



شکل (4-6) داشبورد وب

به منظور فراهم کردن یک رابط کاربری برای مشاهده وضعیت حسگرها، دریافت رویداد های هشدار و ارسال فرمان های کنترلی به سیستم، یک داشبورد تحت وب برای سامانه طراحی و پیاده سازی شده است. این داشبورد به صورت مستقیم با MQTT Broker ارتباط برقرار می کند و برای تبادل داده میان رابط کاربری و برد ESP32 از پروتکل MQTT over WebSocket استفاده می شود.

4-10-2- بخش تنظیمات ارتباط با MQTT

یکی از قسمت های اصلی داشبورد، بخش تنظیمات ارتباط با MQTT Broker است. کاربر می تواند پارامتر های مورد نیاز برای برقراری ارتباط را در این بخش وارد کند. داشبورد برای اتصال به Broker از دو حالت WebSocket و WebSocketSecure پشتیبانی می کند.

4-10-3- دریافت و پردازش داده های حسگر ها

داده های حسگرها توسط ESP32 در Topic های مشخص منتشر می شوند. هنگامی که یک پیام MQTT دریافت می شود، ابتدا Topic آن بررسی شده و بر اساس Topic، پیام به کارت حسگر متناظر فرستاده شده و نمایش داده می شود. برای افزایش قابلیت تحلیل داده، برای حسگر های عددی نمودار تغییرات نیز در نظر گرفته شده است. برای جلوگیری از نمایش داده هایی که ممکن است مربوط به گذشته باشند، داشبورد زمان آخرین دریافت اطلاعات هر حسگر را ثبت می کند. اگر بیش از ۳۰ ثانیه از آخرین پیام یک حسگر گذشته باشد، کارت مربوطه به عنوان داده قدیمی علامت گذاری می شود.

4-10-4- ارسال فرمان های کنترلی

داشبورد علاوه بر دریافت داده، امکان ارسال فرمان به ESP32 را نیز فراهم می کند. برای فرمان های دریافت و ارسال و حذف و نمایش سیگنال های IR و RF می توان از بخش فرمان های داشبورد استفاده کرد. همچنین امکان فعال یا غیرفعال کردن تشخیص حرکت و قطع و وصل کردن سیستم هشدار دستگاه می توان از داشبورد استفاده کرد.

4-10-5- سیستم هشدار داشبورد

برای نمایش رویداد های مهم، یک نوار هشدار در قسمت بالایی داشبورد طراحی شده است. پیام های مربوط سیستم تشخیص حرکت و اعلام دود و حریق در این قسمت نمایش داده می شود.

4-10-6- ذخیره تنظیمات داشبورد

تنظیمات اتصال به Broker وارد شده توسط کاربر، در localStorage مرورگر ذخیره می شوند. این روش باعث می شود کاربر برای هر بار اجرا کردن داشبورد، نیازی به وارد کردن اطلاعات نداشته باشد.

فصل 5:

آزمون و ارزیابی سیستم

5-1- مقدمه

پس از طراحی و پیاده سازی بخش های سخت افزاری و نرم افزاری سامانه، لازم است عملکرد سیستم در شرایط مختلف مورد آزمایش و ارزیابی قرار گیرد. هدف از انجام آزمون ها، بررسی صحت عملکرد اجزای مختلف سامانه، ارزیابی ارتباط میان ماژول های سخت افزاری و نرم افزاری است.

5-2- روش آزمون و ارزیابی سامانه

5-2-1- محیط و تجهیزات مورد استفاده

آزمون سامانه بر روی برد توسعه ESP32 DevKit انجام شده است. در بخش حسگرها، از حسگر DHT11 برای اندازه گیری دما و رطوبت، ماژول LDR برای اندازه گیری شدت نور، خروجی آنالوگ سنسور MQ-2 برای تشخیص دود و گاز های قابل اشتعال و از ماژول PIR برای تشخیص حرکت استفاده شده است. همچنین از یک Active Buzzer و فرستنده و گیرنده RF در دو بلند فرکانسی ۳۱۵ و ۴۳۳ مگاهرتز و فرستنده و گیرنده IR استفاده شده است.

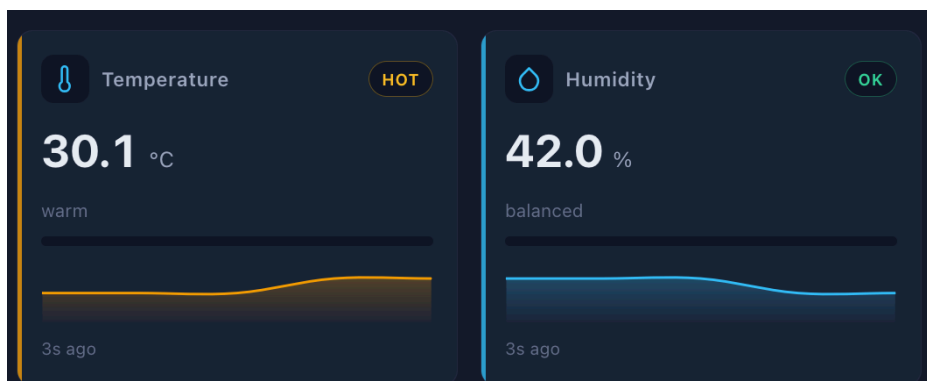
5-2-2- معیار های ارزیابی

برای ارزیابی عملکرد سامانه، معیار های زیر در نظر گرفته شده است:

- صحت دریافت اطلاعات از حسگر های محیطی
- صحت تشخیص شرایط هشدار
- صحت عملکرد سیستم هشدار صوتی
- صحت برقراری ارتباط Wi-Fi
- صحت برقراری ارتباط با MQTT Broker
- صحت دریافت فرمان های کنترلی
- صحت دریافت، ذخیره و ارسال سیگنال های RF
- صحت دریافت، ذخیره و ارسال سیگنال های IR
- صحت عملکرد سیستم Provisioning
- صحت عملکرد داشبورد وب

5-3- آزمون عملکرد حسگرها

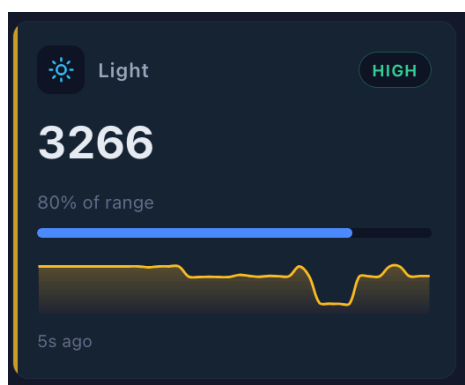
5-3-1 آزمون حسگر دما و رطوبت



شکل (5-1) نتایج حسگر دما و رطوبت

برای بررسی عملکرد حسگر DHT11، داده های دما و رطوبت به صورت دوره ای از طریق توابع مربوط به این حسگر دریافت می شوند. همانطور که مشاهده می شود این حسگر به درستی کار می کند.

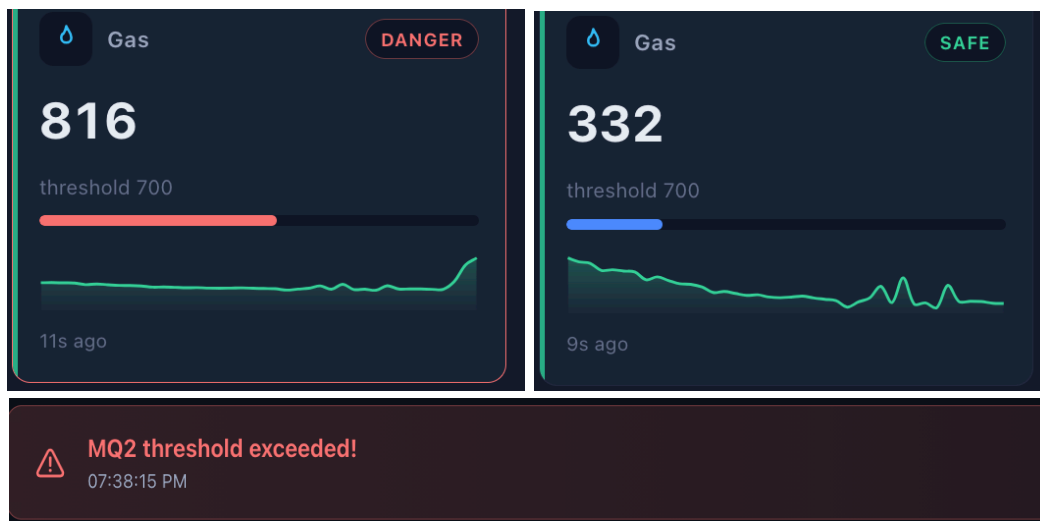
5-3-2 آزمون حسگر نور



شکل (5-2) نتیجه حسگر نور

برای بررسی عملکرد حسگر LDR، شرایط نوری محیط تغییر داده شده و رفتار مقدار خروجی LDR بررسی می شود. انتظار می رود با تغییر شدت نور، مقدار خوانده شده از ورودی آنالوگ نیز تغییر کند. همانطور که مشاهده می شود این حسگر به درستی کار می کند و با تغییر نور محیط مقادیر متفاوت خروجی می دهد.

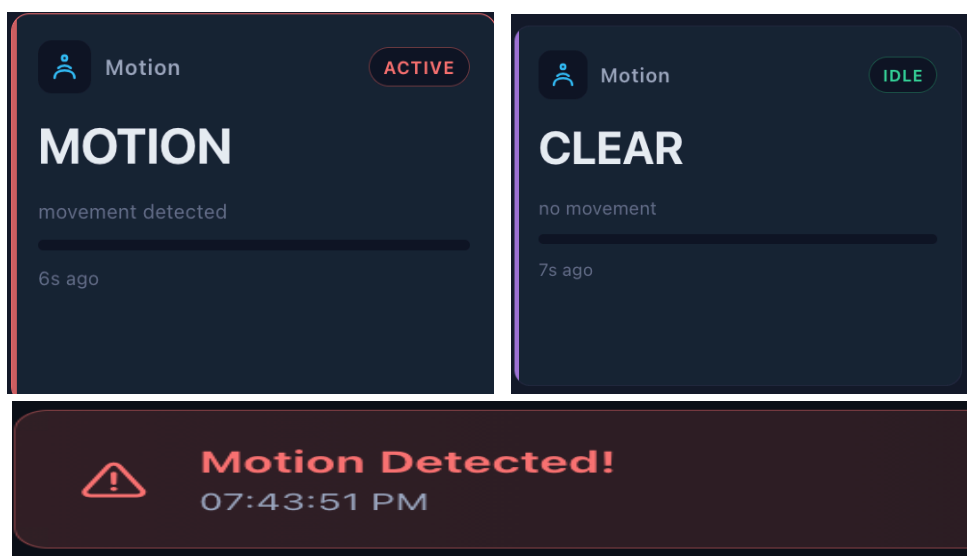
5-3-3- آزمون حسگر دود و آتش



شکل (5-3) نتایج حسگر دود و آتش

در آزمون این بخش، ابتدا دستگاه در شرایط عادی قرار گرفته و مقدار خروجی بررسی می شود. سپس در شرایط دود و آتش قرار گرفته و واکنش سیستم نسبت به افزایش مقدار سنسور ارزیابی می شود. همانطور که مشاهده می شود این حسگر در شرایط عادی عددی در بازه ۳۰۰ تا ۳۵۰ و در شرایط آتش عددی بالاتر از ۷۰۰ که به عنوان آستانه آتش در نظر گرفته شده است می دهد. همچنین مشاهده می شود که در زمان آتش، وضعیت هشدار را فعال کرده و پیام مربوطه را به عنوان هشدار ارسال می کند.

5-3-4- آزمون حسگر تشخیص حرکت



شکل (5-4) نتایج حسگر تشخیص حرکت

برای بررسی عملکرد حسگر، حسگر ابتدا در شرایط عادی قرار داده می شود و سپس یک شی در جلوی آن، تکان داده می شود و واکنش سیستم ارزیابی می شود. همانطور که مشاهده می شود این حسگر به درستی حرکت را تشخیص می دهد و در صورتی که قابلیت تشخیص حرکت سیستم فعال باشد، وضعیت هشدار را فعال کرده و پیام مربوطه را ارسال می کند.

4-5- آزمون سامانه هشدار

در این آزمون، عملکرد Buzzer مورد بررسی قرار می گیرد. در صورت فعال شدن وضعیت هشدار، Buzzer به صورت متناوب فعال و غیر فعال می شود و در صورت غیرفعال شدن وضعیت هشدار، Buzzer غیرفعال می شود. Buzzer در این آزمون به درستی عمل کرده و در زمان هشدار فعال شده و با غیر فعال کردن هشدار، Buzzer نیز غیرفعال می شود.

4-5- آزمون اتصال به Wi-Fi و MQTT Broker

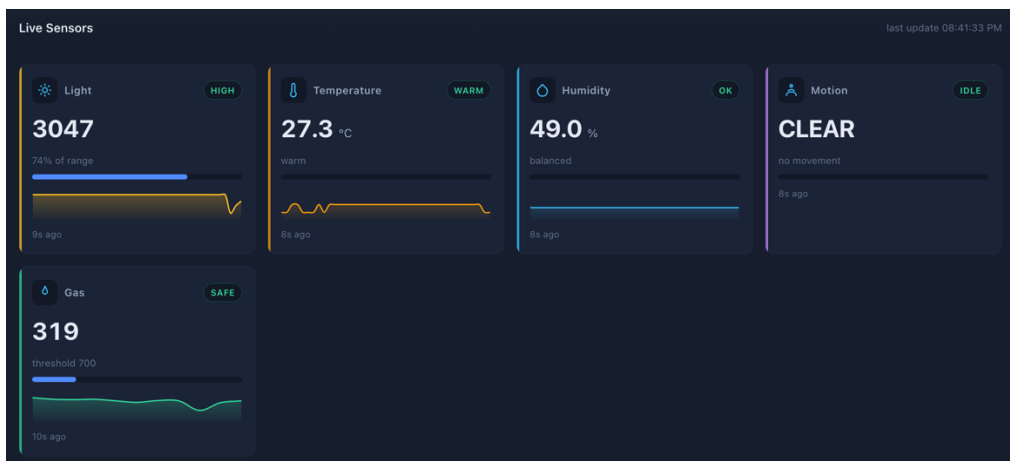
```
Connecting WiFi to Saman's S24 Ultra
Connecting MQTT to broker.emqx.io:1883
MQTT subscribed: homeintelligence/Room/command
```

شکل (5-5) نتیجه آزمون اتصال به شبکه و MQTT Broker

در این آزمون، عملکرد اتصال به Wi-Fi و MQTT Broker و همچنین ارسال و دریافت پیام مورد بررسی قرار می گیرد. همانطور که مشاهده می شود، دستگاه به درستی به شبکه Wi-Fi و سپس به MQTT Broker متصل می شود. ارسال پیام و دریافت پیام از Broker نیز به درستی کار می کند.

4-5- آزمون داشبورد سامانه

در این آزمون عملکرد بخش های مختلف داشبورد مورد بررسی قرار می گیرد.



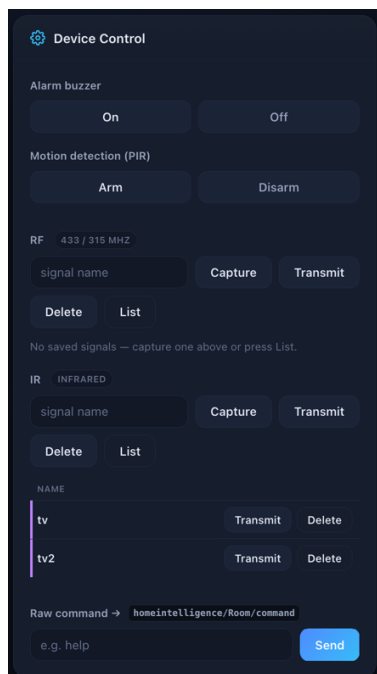
شکل (5-6) نمایش اطلاعات حسگرها در داشبورد

در ابتدا، بخش نمایش اطلاعات حسگرها مورد بررسی قرار گرفته می شود که به درستی کار می کند.



شکل (5-7) نمایش هشدار در داشبورد

سپس، بخش نمایش وضعیت هشدار مورد بررسی قرار گرفته می شود که به درستی کار می کند.



شکل (5-8) ارسال فرمان از داشبورد

و در آخر، بخش ارسال فرمان از داشبورد به دستگاه مورد بررسی قرار گرفته می شود که به درستی کار می کند.

5-7- RF

5-7-1 RF

```
MQTT command on homeintelligence/Room/command: rf rx test
Waiting for RF signal named: test
Saved RF signal: test (433 MHz)
```

شکل (5-9) دریافت سیگنال RF

در این آزمون دریافت سیگنال از ریموت مورد بررسی قرار می گیرد. همانطور که مشاهده می شود، سیگنال به صورت درست از ریموت RF دریافت شده و با نام داده شده ذخیره می شود.

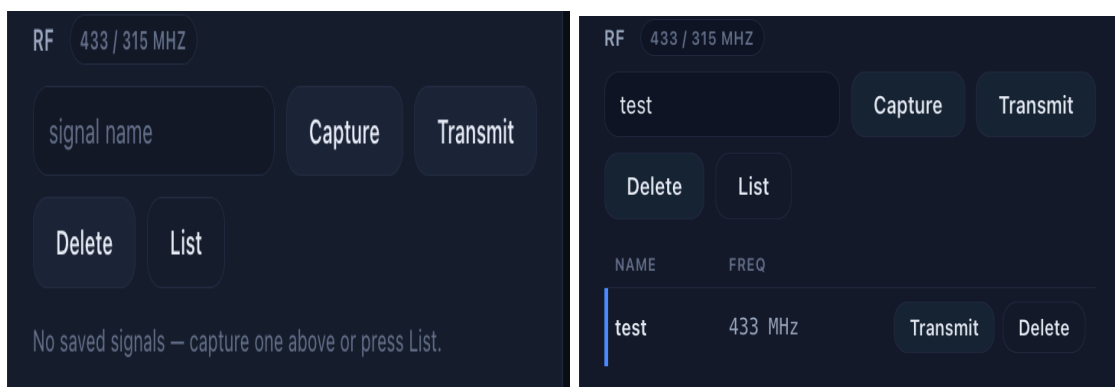
5-7-2 RF

```
MQTT command on homeintelligence/Room/command: rf tx test
Transmitted RF signal: test
```

شکل (5-10) ارسال سیگنال RF

در این آزمون ارسال سیگنال ذخیره شده مورد بررسی قرار می گیرد. همانطور که مشاهده می شود، سیگنال ذخیره شده به درستی ارسال می شود.

5-7-3 RF



```
MQTT command on homeintelligence/Room/command: rf del test
Deleted RF signal: test
```

شکل (5-11) مدیریت سیگنال RF

در این آزمون مدیریت سیگنال ها شامل فهرست کردن تمام سیگنال ها و حذف یک سیگنال مشخص مورد بررسی قرار می گیرد. همانطور که مشاهده می شود، فهرست سیگنال ها به درستی نمایش داده می شود و همچنین پس از حذف یک سیگنال، آن از فهرست حذف می شود.

5-8- آزمون سیستم IR

5-8-1- آزمون دریافت سیگنال IR

```
MQTT command on homeintelligence/Room/command: ir rx test
Waiting for IR signal named: test
NEC
1068E11E
32
68
Saved IR signal: test
```

شکل (5-12) دریافت سیگنال IR

در این آزمون دریافت سیگنال از کنترل IR مورد بررسی قرار می گیرد. همانطور که مشاهده می شود، سیگنال به صورت درست از کنترل IR دریافت شده و با نام داده شده ذخیره می شود.

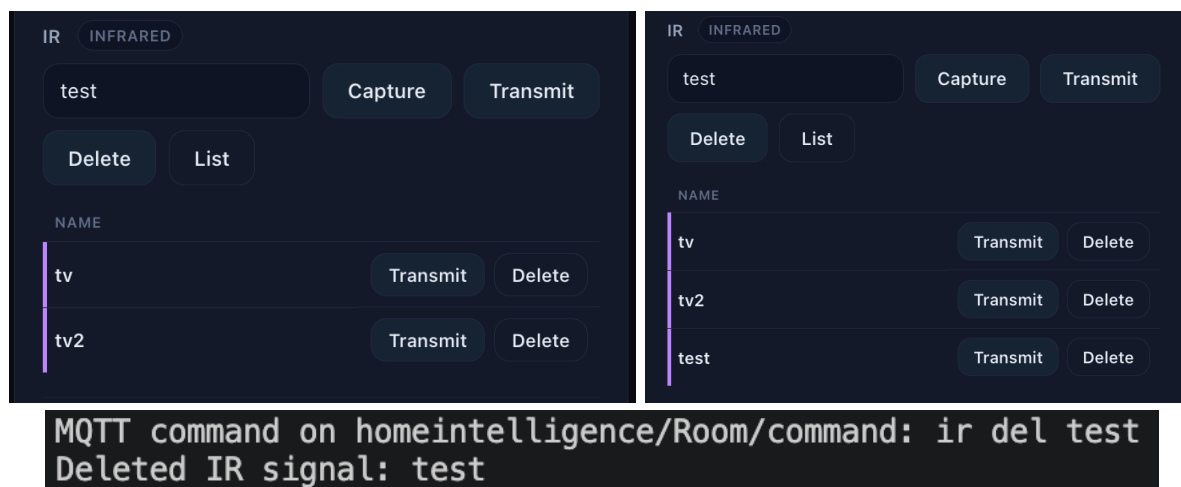
5-8-2- آزمون ارسال سیگنال IR

```
MQTT command on homeintelligence/Room/command: ir tx test
Transmitting NEC signal:
Transmitted IR signal: test
```

شکل (5-13) ارسال سیگنال IR

در این آزمون ارسال سیگنال ذخیره شده مورد بررسی قرار می گیرد. همانطور که مشاهده می شود، سیگنال ذخیره شده به درستی ارسال می شود.

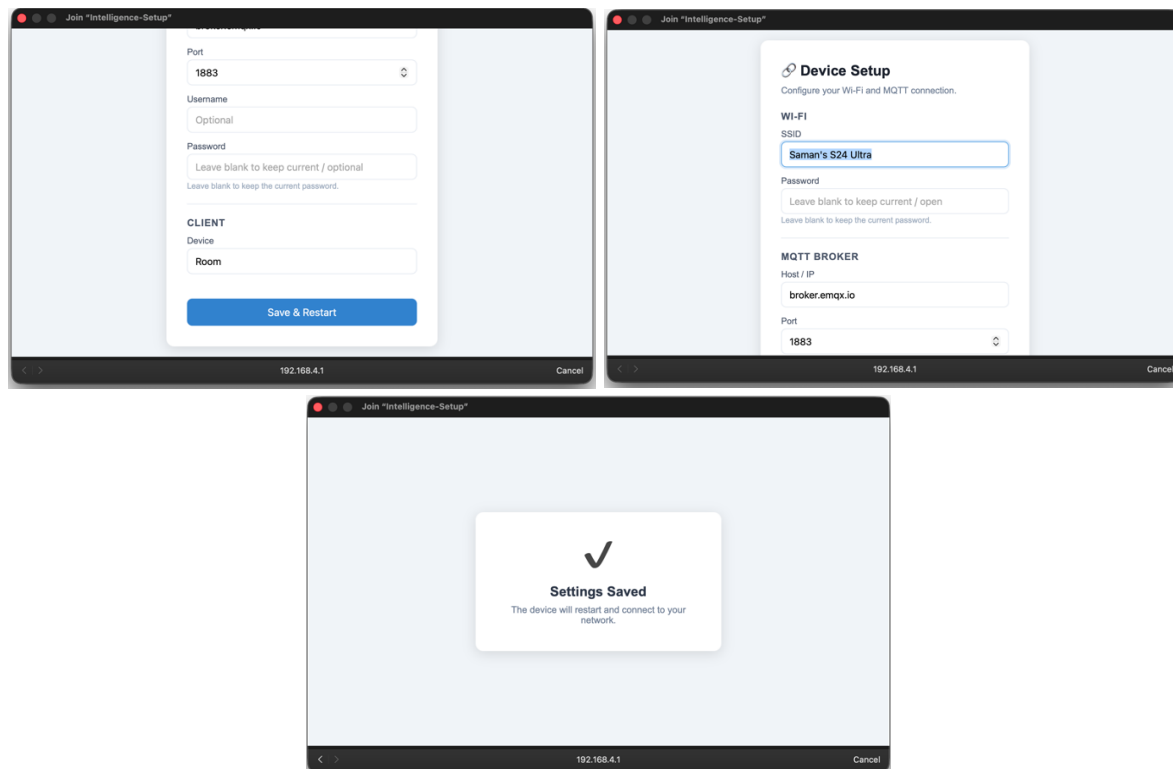
3-8-5- آزمون مدیریت سیگنال IR



شکل (5-14) مدیریت سیگنال IR

در این آزمون مدیریت سیگنال ها شامل فهرست کردن تمام سیگنال ها و حذف یک سیگنال مشخص مورد بررسی قرار می گیرد. همانطور که مشاهده می شود، فهرست سیگنال ها به درستی نمایش داده می شود و همچنین پس از حذف یک سیگنال، آن از فهرست حذف می شود.

5-9- آزمون زیرسامانه Provisioning



شکل (5-15) صفحه تنظیمات Provisioning

در این آزمون عملکرد زیرسامانه Provisioning مورد بررسی قرار می گیرد. همانطور که مشاهده می شود، صفحه تنظیمات پس از اتصال به صورت خودکار نمایش داده می شود و پس از وارد کردن اطلاعات، تنظیمات ذخیره شده و ESP راه اندازی مجدد می شود.

5-10- جمع بندی فصل

در این فصل، عملکرد سیستم توسعه یافته در شرایط مختلف مورد آزمون و بررسی قرار گرفت. نتایج این آزمون ها مبنایی برای ارزیابی میزان تحقق اهداف پروژه و شناسایی نقاط قوت و ضعف سامانه را فراهم می کنند.

فصل 6:

نتیجه گیری

1-6- مقدمه

در این فصل، نتایج حاصل از طراحی، پیاده سازی و ارزیابی سامانه هوشمند تشخیص دود و آتش و کنترل تجهیزات الکتریکی ارائه می شود. هدف اصلی این پروژه، طراحی و توسعه یک سامانه مبتنی بر اینترنت اشیا بوده است که بتواند علاوه بر پایش برخی پارامترهای محیطی، شرایط غیرعادی مانند افزایش میزان گاز و دود یا تشخیص حرکت را شناسایی کرده و در صورت نیاز، هشدار مناسب را فعال کند. همچنین با استفاده از ارتباطات مادون قرمز (IR) و فرکانس رادیویی (RF)، امکان دریافت، ذخیره و ارسال مجدد فرمان های کنترل از راه دور فراهم شده است.

2-6- جمع بندی مراحل انجام پروژه

در این پروژه ابتدا نیازمندی های سامانه بررسی شد. بر اساس این نیازمندی ها، یک معماری سخت افزاری و نرم افزاری مبتنی بر ESP32 طراحی گردید. در بخش سخت افزار، حسگر DHT11 برای اندازه گیری دما و رطوبت، ماژول LDR برای اندازه گیری شدت نور، حسگر MQ-2 برای اندازه گیری مقدار دود و گازهای قابل اشتعال و حسگر PIR برای تشخیص حرکت در بخش ارتباطی سخت افزار، از ماژول های RXB61 و RXB35 برای گرفتن سیگنال های رادیویی و از دو ماژول H34S برای ارسال سیگنال های رادیویی در باند فرکانسی ۳۱۵ و ۴۳۳ مگاهرتز استفاده شد. همچنین از گیرنده مادون قرمز و LED مادون قرمز برای ارتباطات مادون قرمز استفاده شد. در بخش سخت افزار، برنامه به صورت ماژولار طراحی شده و وظایف مختلف میان ماژول ها تقسیم شده اند. این ساختار موجب شده است که توسعه و نگهداری نرم افزار نسبت به یک برنامه یکپارچه ساده تر باشد.

3-6- نتایج حاصل از پیاده سازی

نتایج پیاده سازی نشان می دهد که سامانه توانسته است مجموعه ای از وظایف مورد نظر پروژه را در یک ساختار واحد پیاده سازی کند. قابلیت های حاصل شده شامل موارد زیر است:

- دریافت اطلاعات دما و رطوبت از حسگر DHT11
- دریافت مقدار شدت نور از حسگر LDR
- دریافت مقدار دود و گازهای قابل اشتعال از حسگر MQ2
- تشخیص وضعیت حرکت توسط حسگر PIR
- فعال سازی هشدار صوتی در شرایط خطر
- دریافت، ذخیره و ارسال سیگنال های رادیویی در دو باند فرکانسی ۳۱۵ و ۴۳۳ مگاهرتز
- دریافت، ذخیره و ارسال سیگنال های مادون قرمز

- اتصال به شبکه Wi-Fi و MQTT Broker
- ارسال داده های حسگر ها به داشبورد
- دریافت فرمان های کنترلی از داشبورد

4-6- محدودیت های پروژه

با وجود پیاده سازی قابلیت های اصلی، سامانه دارای محدودیت هایی است که باید مورد توجه قرار گیرند. نخست، مقدار خروجی MQ-2 در این پروژه به صورت مستقیم و با یک مقدار آستانه نرم افزاری بررسی می شود. بنابراین این مقدار به شرایط محیطی، زمان گرم شدن حسگر و ویژگی های فیزیکی آن وابسته است و برای استفاده به عنوان سامانه استاندارد اعلام حریق نیازمند کالیبراسیون و بررسی های تخصصی بیشتری خواهد بود.

دوم، قابلیت RF و IR به سازگاری تجهیزات موردنظر با نوع سیگنال های قابل دریافت و ارسال توسط سخت افزار و کتابخانه های استفاده شده وابسته است. بنابراین نمی توان تضمین کرد که تمام تجهیزات موجود در بازار با این سامانه قابل کنترل باشند.

همچنین Broker مورد استفاده در پیاده سازی نهایی، یک Broker عمومی است. بنابراین برای استفاده در محیط های واقعی و حساس، استفاده از زیرساخت اختصاصی و اعمال سازوکار های امنیتی مانند احراز هویت و رمزنگاری ارتباط توصیه می شود.

4-5-6- پیشنهاد های توسعه در آینده

1-5-6- افزایش امنیت ارتباطات

در نسخه های آینده می توان ارتباط MQTT را به MQTT over TLS و برای Broker احراز هویت مناسب تعریف نمود.

2-5-6- استفاده از Broker اختصاصی

به جای استفاده از Broker عمومی، می توان یک Broker اختصاصی در سرور با زیرساخت ابری مستقر کرد.

3-5-6- توسعه قابلیت OTA

اضافه کردن قابلیت Over-The-Air (OTA) می تواند امکان بروزرسانی Firmware دستگاه را بدون اتصال فیزیکی ESP32 به رایانه فراهم کند.

4-5-6- اضافه کردن ارتباطات جایگزین

در نسخه های آینده می توان در کنار Wi-Fi از فناوری هایی مانند GSM، LoRa یا سایر روش های ارتباطی استفاده کرد تا در صورت قطع شبکه Wi-Fi، مسیر جایگزینی برای ارسال هشدار وجود داشته باشد.

4-6- جمع بندی نهایی

در مجموع، نتایج پروژه نشان می دهد که می توان با استفاده از یک برد کم هزینه و ترکیب حسگر های محیطی، ارتباط MQTT و قابلیت های RF و IR، یک هاب هوشمند چندمنظوره برای پایش و کنترل محیط ایجاد کرد. با این حال، برای تبدیل این نمونه دانشگاهی به یک محصول قابل استفاده در محیط های واقعی، انجام کالیبراسیون دقیق حسگر ها، افزایش امنیت ارتباطات و استفاده از زیرساخت ارتباطی اختصاصی و انجام آزمون های استاندارد ضروری خواهد بود.

فصل 7:

مراجع

Bibliography

- [1] National Institute of Standards and Technology (NIST), "IoT Device," 2024. [Online]. Available: https://csrc.nist.gov/glossary/term/iot_device. [Accessed August 2026].
- [2] J. Voas, "Networks of 'Things'," National Institute of Standards and Technology, NIST Special Publications 800-183, 2016.
- [3] Espressif Systems, "ESP32 series Datasheet," Espressif Systems.
- [4] A. Banks, E. Briggs, K. Borgendale and R. Gupta, "MQTT Version 5.0," OASIS Standards, 2019.
- [5] S. Ullrich, "rc-switch: Arduino library to operate 315/433 MHz remote controlled devices," [Online]. Available: <https://github.com/sui77/rc-switch>. [Accessed August 2026].
- [6] D. Conran et al., "IRremoteESP8266: Infrared remote library for ESP8266/ESP32," [Online]. Available: <https://github.com/crankyoldgit/IRremoteESP8266>. [Accessed August 2026].
- [7] Hanwei Electronics Group Corporation, "MQ-2 Gas Sensor Datasheet," Hanwei Electronics Group Corporation.
- [8] Adafruit Industries, "DHT Sensor Library Documentation," [Online]. Available: <https://adafruit.github.io/DHT-sensor-library/>. [Accessed August 2026].
- [9] Espressif Systems, "Arduino ESP32 Wi-Fi Documentation," Espressif Systems.
- [10] Arduino ESP32, "LittleFS File System Documentation," Espressif Systems.

فصل 8:

پیوست‌ها

8-1- نقشه شماتیک مدار

